



Master's Degree Program

Field of Study: Advanced Analysis – Big Data

Thi Van Trang Le

Number ID : 140036

Process Intelligence with Python Language

Master's Thesis

Under the supervision of

dr hab. Michała Ramsza

Institute of Mathematical Economics

Warszawa 2026

Contents

1	Introduction	5
1.1	Background and motivation	5
1.2	Research Objectives and Research Question	6
1.3	Relevance and Academic Context	6
1.4	Methodology	7
1.5	Structure of the thesis	8
2	The mathematics of process mining	10
2.1	Event logs and traces	10
2.2	Activities, variants, and directly-follows relations	12
2.3	Process discovery and process models	13
2.3.1	Petri nets	14
2.3.2	The Alpha algorithm	15
2.3.3	The Inductive Miner and process trees	16
2.4	Conformance checking	17
2.5	Performance analysis and process intelligence	19
2.6	Event data representation in Python	20
2.7	Exploratory description of the BPI Challenge 2019 dataset	21
3	The tools available in Python	24
3.1	Core process mining libraries	24
3.1.1	PM4Py	24
3.1.2	PM4Py-Streaming	26
3.1.3	Simod	26
3.1.4	PMLab	27
3.1.5	BupaR (Python interface)	27
3.2	Predictive and API-oriented tools	27
3.2.1	ML4ProM	28
3.2.2	PM4Py-ML	28
3.2.3	PyCelonis	29
3.2.4	BPMN-Python and workflow automation	29
3.2.5	Predictive Process Monitoring Toolkit (PPMT)	29
3.3	Simulation and optimization tools	30
3.3.1	SimPy	30
3.3.2	BPSim	30
3.3.3	Process Optimizer	31
3.4	Visualization libraries	31
3.4.1	Graphviz	31
3.4.2	Plotly and Dash	32
3.4.3	NetworkX	32
3.4.4	Pyvis	32
3.5	Complementary data processing libraries	33
3.5.1	Pandas	33

3.5.2	NumPy	33
3.5.3	Scikit-Learn	33
3.5.4	XGBoost and LightGBM	33
3.5.5	TensorFlow and PyTorch	34
3.6	Academic and community resources	34
3.7	Summary	34
4	Worked examples	36
4.1	Introductory synthetic example	36
4.2	BPI Challenge 2019: the case study	41
4.2.1	Data source, format, and structure	41
4.2.2	Purpose and analytical workflow	43
4.2.3	Implementation and results	45
4.2.4	Discussion of Results	49
4.3	Advanced Example: Loops, Variants, and Performance Analysis	51
4.3.1	Variant Analysis	51
4.3.2	Loop and Rework Detection	52
4.3.3	Throughput Time and Performance KPIs	53
4.3.4	Implementation Overview	55
5	Conclusions	61
5.1	Summary of Findings	61
5.2	Limitations	62
5.3	Directions for Future Research	62
	Bibliography	64
	List of figures	65
	List of tables	66

Chapter 1

Introduction

1.1 Background and motivation

Today, nearly every business process is managed through digital systems — whether an ERP platform, a CRM tool, or a workflow application. Each action, from approvals and payments to status updates, is automatically logged: capturing who performed it, when, and for which specific case. While these records may seem routine on their own, together they provide a detailed picture of how work actually gets done — which often differs significantly from how processes were originally designed.

By analyzing this data, organizations gain a much clearer and more accurate understanding of their daily operations. Unlike traditional management reports that describe how things are supposed to work, process mining examines the actual sequence of events by tracing the digital footprints left in the system. This approach reveals where bottlenecks or repeated work truly occur and highlights variations between teams, locations, or individual cases. In many instances, these insights expose surprising gaps between the intended process and what actually takes place.

Since the late 1990s, when van der Aalst and his colleagues introduced the first process mining algorithms, the field has grown considerably. What began as an academic pursuit has since become an essential tool for businesses worldwide. Dedicated conferences, an official IEEE task force, and a wide range of practical applications in industries such as healthcare, finance, logistics, and manufacturing all underscore the growing value of process mining in both research and everyday business practice.

Process intelligence takes process mining a step further by shifting from simply analyzing past events to proactively shaping the future. Instead of just looking back at what has already happened, process intelligence involves real-time monitoring of ongoing cases, forecasting where issues might develop, and running simulations to see how changes—like reallocating resources—could impact results. Ultimately, the aim is to turn insights from historical data into practical guidance that helps organizations make smarter decisions as their processes play out.

Python has become the leading language for process mining and process intelligence work. Its widespread adoption in the data science community, combined with a growing collection of specialized libraries, makes it a natural choice for practitioners. PM4Py, for example, offers a comprehensive suite of tools — from loading event logs and discovering process models to checking conformance and analyzing performance. Additional libraries provide capabilities for simulation, prediction, and visualization, making it possible to conduct an entire process intelligence project within the Python ecosystem.

Despite these advances, navigating the Python process mining ecosystem can still be challenging. Many libraries have overlapping features, documentation quality varies considerably, and no single resource clearly explains which tools are best suited for

which tasks or how to apply them in real-world scenarios. This thesis is designed to address that gap by providing a structured overview of the key tools available, together with practical code examples that readers can adapt for their own projects.

1.2 Research Objectives and Research Question

The central question driving this thesis is:

Which Python tools are available for process mining and process intelligence, and how well do they support key analytical tasks such as process discovery, conformance checking, and performance analysis?

To answer this question, the thesis sets out four specific objectives:

1. To introduce and explain the mathematical concepts underpinning process mining — such as event logs, traces, directly-follows relations, process models, and conformance measures — to enable a rigorous comparison of different tools.
2. To review the main Python libraries for process mining and process intelligence, organizing them by primary function and describing their features, strengths, and limitations.
3. To demonstrate how selected Python tools can be applied in real-world scenarios, using examples that range from simple synthetic datasets to large procurement event logs, and extending beyond basic process discovery to include performance measurement, simulation, and outcome prediction.
4. To draw conclusions about the current Python process mining ecosystem, highlighting gaps and suggesting directions for future development.

This thesis focuses on Python-based tools and the three core process mining tasks — discovery, conformance checking, and performance analysis — as well as related capabilities such as simulation and predictive monitoring. The emphasis is on open-source solutions accessible to both practitioners and researchers without the need for commercial licenses. The only exception is PyCelonis, which is included to represent an important category of commercial process intelligence platforms that offer a Python interface.

1.3 Relevance and Academic Context

Process mining emerged as a research field in the late 1990s and early 2000s, thanks in large part to the pioneering work of Wil van der Aalst and his colleagues at Eindhoven University of Technology. Since then, the field has grown significantly, now boasting its own annual conference — the International Conference on Process Mining (ICPM) — and a formal IEEE task force that has established clear principles and goals for the discipline. The textbook by van der Aalst (Aalst, 2022) remains the definitive reference for both theoretical and practical aspects of process mining and serves as the primary academic resource for this thesis.

A major driver of progress in the field has been the annual BPI Challenge, held alongside ICPM. This competition provides researchers with real-world datasets from organizations and invites them to apply process mining techniques. These datasets have become standard benchmarks, widely used to compare algorithms and tools. For this thesis, the BPI Challenge 2019 dataset serves as the main case study. Drawn from a large multinational company’s procurement process, it includes nearly 1.6 million events and is recognized as one of the most complex datasets in the series. Its broad use in academic studies makes it an excellent foundation for demonstrating process mining methods in a realistic setting.

The Python ecosystem for process mining has evolved rapidly since the introduction of PM4Py in 2018. Earlier implementations such as ProM, which are Java-based, remain prevalent in academic circles but require more specialized expertise to extend and deploy. The rise of Python-based libraries has made it considerably easier for both researchers and practitioners to engage with the field, and has enabled process mining to integrate more naturally with other data science tools for machine learning, visualization, and statistical analysis. New libraries continue to appear, supporting advanced tasks such as streaming process mining, simulation, and predictive monitoring. Nevertheless, the Python ecosystem remains less mature and less thoroughly documented than its Java-based counterpart, making a clear and structured review especially valuable at this stage.

This thesis contributes a comprehensive, practice-oriented review of the Python process mining ecosystem, with a special focus on PM4Py and its related extensions. All practical examples are provided as Jupyter notebooks, ensuring that others can reproduce and verify the results. This emphasis on reproducibility is important, as it allows the findings presented in 4 to be validated and further developed by future researchers and practitioners.

1.4 Methodology

This thesis combines a structured review with empirical demonstration and is organized into four sequential phases.

Phase 1: Theoretical foundation. The mathematical concepts essential to process mining are introduced and formalized in 2. The main source for these definitions is van der Aalst (Aalst, 2022). Establishing this foundation first provides a common vocabulary for comparing tools and interpreting the results of practical examples.

Phase 2: Tool survey. Python libraries are identified using academic sources — such as papers and conference proceedings from ICPM — the Python Package Index (PyPI), GitHub repositories, and cross-references within the PM4Py documentation. For each tool, the analysis covers the primary analytical task supported, accepted data formats, implemented algorithms, and the level of active maintenance and community support, as reflected by recent commit history and documentation quality. Tools are grouped into five categories: core process mining libraries, predictive and AI-oriented

tools, simulation and optimization tools, visualization libraries, and complementary data processing packages.

Phase 3: Practical implementation. Three Jupyter notebooks are developed to demonstrate the end-to-end process intelligence workflow. The first notebook uses a small, hand-built dataset to introduce core process mining steps in a controlled setting. The remaining two notebooks use the BPI Challenge 2019 procurement dataset and progressively expand the analysis: process discovery and conformance checking; and variant analysis, throughput time measurement, and rework detection. Each notebook contains executable code, outputs, and saved figures, ensuring full reproducibility.

Phase 4: Synthesis and conclusions. The results from the tool survey and practical examples are combined in 5, where the research question is addressed, limitations of the work are acknowledged, and future research directions are proposed.

The methodology is descriptive and demonstrative rather than strictly evaluative; no formal benchmarking is performed. The thesis does not compare runtime, fitness scores, or precision values of discovery algorithms on the same data, as such experiments would require a controlled design beyond the scope of this work. The contribution lies in the structured explanation, organization, and demonstration of available tools.

1.5 Structure of the thesis

The remainder of this thesis is organized as follows.

Mathematical Foundations (2). This chapter introduces the mathematical foundations of process mining. It formally defines events, traces, and event logs; explains directly-follows relations and their importance in process discovery; describes key process model representations used in the thesis — the directly-follows graph, process tree, and Petri net — and introduces conformance checking and performance analysis. The chapter also covers the Object-Centric Event Log (OCEL) format and provides an exploratory overview of the BPI Challenge 2019 dataset.

Tool Review (3). This chapter reviews the main Python tools available for process mining and process intelligence. Tools are organized into five categories: core process mining libraries, predictive and AI-oriented tools, simulation and optimization tools, visualization libraries, and complementary data processing packages. Each category is described in terms of the tasks it supports and the tools it contains, and the chapter concludes with a comparative summary table.

Practical Examples (4). This chapter walks through three practical Jupyter notebooks that together cover the core process mining workflow. The first notebook introduces process mining concepts using a synthetic event log. The second applies these concepts to real procurement data from the BPI Challenge 2019, producing a Petri net model, a conformance score, and activity frequency analysis. The third notebook extends the analysis with variant analysis, throughput time distribution, and rework detection.

Conclusions (5). The final chapter summarizes the main findings of the thesis, discusses its limitations, and suggests directions for future research.

Chapter 2

The mathematics of process mining

A solid understanding of process mining begins with a shared vocabulary. Without precise definitions, terms like “fitness” or “conformance” can mean different things to different people, making it nearly impossible to compare tools or algorithms effectively. This chapter introduces the core mathematical concepts used throughout the thesis: events, traces, and event logs; directly-follows relations and variants; process discovery and process models; conformance checking; and performance analysis. Each concept is paired with a small, concrete example to bridge the gap between formal definition and practical application. The chapter also shows how event data are represented in Python and provides an initial exploration of the BPI Challenge 2019 dataset used in the case study in 4.

The definitions and notation in this chapter closely follow van der Aalst (Aalst, 2022).

2.1 Event logs and traces

At the heart of process mining is the **event** — the fundamental unit of data. Each event captures a specific activity performed for a particular process instance at a certain time. Formally:

- Let $\mathcal{U}_C$ be the set of all case identifiers, each representing a process instance.
- Let $\mathcal{U}_A$ be the set of all activity names.
- Let $\mathcal{U}_T$ be the set of all timestamps, with a natural chronological ordering $\leq$.

An **event** e is a tuple

$$e = (c, a, t) \in \mathcal{U}_C \times \mathcal{U}_A \times \mathcal{U}_T, \quad (1)$$

where $c \in \mathcal{U}_C$ is the case identifier, $a \in \mathcal{U}_A$ is the activity name, and $t \in \mathcal{U}_T$ is the timestamp. In practice, events can include more information — such as the resource who performed the action, costs, or other business-specific data — but case identifier, activity, and timestamp are the core attributes. When two events in the same case share the same timestamp, their relative order is resolved by a unique event identifier assigned at recording time.

This thesis structures event data according to the XES standard (eXtensible Event Stream): an event log is a collection of cases, each being a sequence of events ordered by timestamp. The Object-Centric Event Log (OCEL) format, which allows events to belong to multiple objects simultaneously, is introduced separately in 2.6.

A **trace** is the projection of all events belonging to a single case onto the sequence of their activity names, ordered by timestamp. If A is a finite set of activity names, a trace σ is written as

$$\sigma = \langle a_1, a_2, \dots, a_n \rangle, \quad (2)$$

where each $a_i \in A$. The set of all finite sequences over A is denoted A^* — the Kleene closure of A . Note that this projection retains only activity names: two cases with identical activity sequences but different timestamps or other attributes are treated as the same trace. This abstraction focuses analysis on control flow, which is the primary concern in process discovery, while deliberately setting aside attribute variation. When attribute-aware analysis is needed, additional event properties are incorporated at a later stage.

A **variant** is a unique trace pattern — a specific sequence of activity names that appears at least once in the log. Two cases belong to the same variant if and only if their traces are identical.

An **event log** L collects traces from many cases. Since multiple cases can follow the same trace, L is modelled as a multiset over A^* :

$$L \in \mathcal{B}(A^*), \quad (3)$$

where $\mathcal{B}$ denotes the multiset operator. For a trace $\sigma \in A^*$, $L(\sigma)$ gives the number of cases in which σ occurs. Multiset notation uses superscripts to indicate multiplicity: $[\sigma_1^2, \sigma_3^1]$ means σ_1 appears twice and σ_3 appears once.

Concrete example. Consider the event log in 1.

Table 1: An example event log with three cases, each consisting of three events.

Case ID	Event ID	Activity	Timestamp
1	e1	Create order	09:00
1	e2	Approve order	10:00
1	e3	Send invoice	11:00
2	e4	Create order	09:15
2	e5	Approve order	10:30
2	e6	Send invoice	11:20
3	e7	Create order	08:45
3	e8	Reject order	09:50
3	e9	Close case	10:40

Source: Own elaboration.

Grouping by case identifier and ordering by timestamp gives three traces:

- Case 1: $\sigma_1 = \langle \text{Create order}, \text{Approve order}, \text{Send invoice} \rangle$
- Case 2: $\sigma_2 = \langle \text{Create order}, \text{Approve order}, \text{Send invoice} \rangle$
- Case 3: $\sigma_3 = \langle \text{Create order}, \text{Reject order}, \text{Close case} \rangle$

Since $\sigma_1 = \sigma_2$, the log is the multiset $L = [\sigma_1^2, \sigma_3^1]$: the approval path is observed twice and the rejection path once. The log has three cases and two distinct variants.

Summary statistics. Before applying discovery algorithms, analysts typically compute several standard statistics. The **number of distinct variants** is

$$|\text{Var}(L)| = |\{\sigma \in A^* : L(\sigma) > 0\}|. \quad (4)$$

The **average trace length** is

$$\bar{\ell} = \frac{\sum_{\sigma \in A^*} L(\sigma) \cdot |\sigma|}{|L|}. \quad (5)$$

The **variant frequency distribution** ranks traces by their multiplicity in descending order. In real-world logs, a small number of variants typically account for the large majority of cases, while many variants appear only once or twice — a pattern that guides filtering and sampling decisions before discovery.

Traces reveal both the structure and variability of a process. If most cases follow the same sequence, the process is stable and predictable; if many different traces appear, the process is more variable and may require a more flexible model. The distribution of trace frequencies is therefore one of the first things analysts examine to gauge the complexity and diversity of a process.

2.2 Activities, variants, and directly-follows relations

A key structural relationship in any event log is the directly-follows relation. Activity b **directly follows** activity a in trace $\sigma = \langle a_1, \dots, a_n \rangle$ if there exists an index i such that $a_i = a$ and $a_{i+1} = b$; this is written $a \overset{\sigma}{>} b$. The directly-follows relation for the entire log is obtained by combining the directly-follows pairs from all distinct traces:

$$\overset{L}{>} := \bigcup_{\sigma \in \text{Set}(L)} \overset{\sigma}{>}, \quad (6)$$

where $\text{Set}(L) = \{\sigma \in A^* : L(\sigma) > 0\}$ is the set of distinct traces appearing in L .

The **frequency** of a directly-follows pair (a, b) tells us how often b comes right after a throughout the log. For each distinct trace σ , $L(\sigma)$ gives the number of cases with that trace, and $|\{i : a_i = a \wedge a_{i+1} = b\}|$ counts how many times the pair (a, b) appears within σ . Summing across all distinct traces gives:

$$f_{L(a,b)} = \sum_{\sigma \in \text{Set}(L)} L(\sigma) \cdot |\{i : a_i = a \wedge a_{i+1} = b\}|. \quad (7)$$

The **directly-follows graph** (DFG) is a weighted directed graph constructed from the log:

$$\text{DFG}(L) = (A_L, E_L, f_L), \quad (8)$$

where A_L is the set of all activities appearing in the log, $E_L = \{(a, b) \in A_L \times A_L : a \overset{L}{>} b\}$ is the set of directed edges, and $f_L : E_L \rightarrow \mathbb{N}$ assigns each edge its frequency. The graph also includes a source node connected to all starting activities and a sink node receiving edges from all ending activities, with edge weights based on how often activities start or end a trace.

Using the event log from 1, the directly-follows pairs and their frequencies are listed in 2.

Table 2: Directly-follows pairs and their frequencies derived from 1.

From activity	To activity	Frequency
Create order	Approve order	2
Approve order	Send invoice	2
Create order	Reject order	1
Reject order	Close case	1

Source: Own elaboration.

Figure 1: Directly-follows graph for the example event log from 1. Node size reflects activity frequency; arc labels show the number of directly-following transitions.



Source: Own elaboration based on 1.

The DFG in 1 shows two main execution paths: the approval path (Create → Approve → Send, frequency 2) and the rejection path (Create → Reject → Close, frequency 1). This already points to an exclusive choice after Create order. The DFG gives a quick visual summary of which activities typically follow one another and how frequently. However, it does not reveal more complex control-flow structures such as sequences, choices, or parallelism, so it is limited for deeper structural analysis.

Variant analysis involves ranking all variants by how often they occur. The **frequency** of a variant σ is $L(\sigma)$ — the number of cases following that exact trace. The **coverage** of the top k variants is:

$$\text{Coverage}(k) = \frac{\sum_{i=1}^k L(\sigma_i^*)}{|L|}, \quad (9)$$

where $\sigma_1^*, \sigma_2^*, \dots$ are the variants ordered by decreasing frequency. Filtering the log to keep only the top- k variants is a common preprocessing step that helps analysts focus on dominant behavior and reduce the effect of rare or exceptional cases. In the BPI Challenge 2019 dataset, the full log contains 26,378 distinct variants, illustrating just how diverse real-world procurement processes can be.

Basic statistics — the number of events per activity, the total number of cases, the count of distinct variants, and the average trace length — provide a useful initial summary of the log before applying more detailed analysis techniques.

2.3 Process discovery and process models

Process discovery is the task of building a process model directly from an event log, without relying on any existing reference model. The goal is to find a model that accurately reflects the real-life behavior captured in the log. The **language** of a model M , denoted $\mathcal{L}(M)$, is the set of all traces that the model can generate when simulated.

A well-discovered model must balance four key quality dimensions:

- **Fitness** checks whether the model can reproduce the traces actually observed in the log. A model has perfect fitness if every observed trace can also be generated by the model: $\text{Var}(L) \subseteq \mathcal{L}(M)$. In practice, perfect fitness is not always ideal — it can lead to overly complex models — and a small loss of fitness is sometimes accepted in exchange for better precision or simplicity.
- **Precision** measures whether the model generates only the behavior seen in the log and avoids allowing traces that never occurred. A model that is too permissive accepts many traces not present in the real data. A degenerate example is a model that allows any activity in any order: it would have perfect fitness but zero precision.
- **Generalization** is about how well the model captures plausible but unseen variations of the process. A model that simply memorizes every trace in the log may fit the current data perfectly but fail to represent valid cases that just did not happen to appear yet. Good generalization means the model can handle realistic new behavior, not only what is in the data.
- **Simplicity** refers to how easy the model is to understand. All else being equal, a simpler model is preferred, provided it still offers reasonable fitness and precision. The appropriate simplicity measure depends on the model type: for a directly-follows graph it is the number of nodes and edges; for a Petri net it is the number of places and transitions.

These four dimensions often pull in different directions. Improving fitness can make a model more permissive, which lowers precision. Allowing for generalization also reduces precision, since more traces — including some not seen in the log — become acceptable. Simplicity tends to restrict the model, which can improve precision but may lower fitness and generalization. Striking the right balance is a central challenge in process discovery, and different algorithms make different trade-offs depending on the log and the analysis goals.

2.3.1 Petri nets

The primary process model representation used in this thesis is the Petri net, which is also the standard output format for PM4Py and most other process mining tools. Formally, a Petri net is defined as a triple $N = (P, T, F)$, where:

- P is a finite set of **places**, drawn as circles,
- T is a finite set of **transitions**, drawn as rectangles, with $P \cap T = \emptyset$,
- $F \subseteq (P \times T) \cup (T \times P)$ is the **flow relation**, specifying the directed arcs between places and transitions.

For any transition $t \in T$, its set of input places is $\text{Pre}(t) = \{p \in P : (p, t) \in F\}$ and its output places are $\text{Post}(t) = \{p \in P : (t, p) \in F\}$.

A **marking** $M : P \rightarrow \mathbb{N}_0$ assigns a non-negative integer number of tokens to each place. A transition t is **enabled** in marking M if $M(p) \geq 1$ for all $p \in \text{Pre}(t)$. When t fires, it produces a new marking M' :

$$M'(p) = \begin{cases} M(p) - 1 & : \text{if } p \in \text{Pre}(t) \text{ and } p \notin \text{Post}(t) \\ M(p) + 1 & : \text{if } p \in \text{Post}(t) \text{ and } p \notin \text{Pre}(t) \\ M(p) & : \text{otherwise.} \end{cases} \quad (10)$$

An **accepting Petri net** (N, M_0, M_f) specifies an initial marking M_0 and a final marking M_f . A trace $\sigma = \langle a_1, \dots, a_n \rangle$ is accepted if firing the corresponding transitions in order leads from M_0 to M_f .

2.3.2 The Alpha algorithm

The Alpha algorithm was the first method to automatically discover a Petri net from an event log. It analyzes the directly-follows relation $\underset{L}{>}$ and classifies each pair of activities into one of four types:

- $a \underset{L}{>} b$ (**direct succession**): b directly follows a in at least one trace in L .
- $a \underset{L}{\gg} b$ (**causality**): $a \underset{L}{>} b$ holds but $b \underset{L}{>} a$ does not — a appears to cause b .
- $a \parallel b$ (**parallelism**): both $a \underset{L}{>} b$ and $b \underset{L}{>} a$ hold — the activities can occur in any order.
- $a \# b$ (**independence**): neither $a \underset{L}{>} b$ nor $b \underset{L}{>} a$ holds — the activities never directly follow one another.

The algorithm then searches for **causal pairs** (X, Y) , where X and Y are non-empty sets of activities such that every $a \in X$ causally leads to every $b \in Y$, and activities within X and within Y are mutually independent. A pair is **maximal** if no activity can be added to X or Y without breaking these properties. Each maximal causal pair becomes a place in the Petri net, with transitions in X as inputs and transitions in Y as outputs.

Example. Consider $L = [\langle A, B, D \rangle^4, \langle A, C, D \rangle^3]$. The directly-follows pairs are $A \underset{L}{>} B$, $A \underset{L}{>} C$, $B \underset{L}{>} D$, and $C \underset{L}{>} D$. The derived relations are: $A \underset{L}{\gg} B$, $A \underset{L}{\gg} C$, $B \underset{L}{\gg} D$, $C \underset{L}{\gg} D$, $B \# C$, and $A \# D$. The maximal causal pairs are $(\{A\}, \{B, C\})$ and $(\{B, C\}, \{D\})$. For instance, $(\{B\}, \{D\})$ and $(\{C\}, \{D\})$ are valid but not maximal, since both are contained within $(\{B, C\}, \{D\})$.

Adding a start place linked to A and an end place receiving from D , the algorithm creates four places (i, p_1, p_2, o) and four transitions (A, B, C, D) . Place p_1 (from $(\{A\}, \{B, C\})$) represents an exclusive choice between B and C ; p_2 (from $(\{B, C\}, \{D\})$) merges both paths before D . This structure correctly models both variants in the log.

The Alpha algorithm was groundbreaking for showing that process models can be automatically discovered from event data. However, it has clear limitations: it struggles with loops, is sensitive to noise and rare behavior, cannot represent all control-flow patterns,

and fails when its underlying assumptions are violated. More advanced algorithms — such as the Inductive Miner — have since been developed to address these issues.

2.3.3 The Inductive Miner and process trees

The Inductive Miner is a more modern and widely used discovery algorithm that overcomes many of the Alpha algorithm’s weaknesses. It works by recursively splitting the event log into smaller sublogs based on detected control-flow patterns — sequences, exclusive choices, parallelism, or loops. Each split corresponds to a node in a **process tree**.

A process tree is a hierarchical model: leaf nodes are activities (or the silent activity τ , representing an invisible step) and internal nodes are operators defining how their children are executed. The four main operators are:

- $\rightarrow (T_1, \dots, T_k)$: execute children left-to-right in order (sequence).
- $\times (T_1, \dots, T_k)$: execute exactly one child (XOR choice).
- $+(T_1, \dots, T_k)$: execute all children in any interleaved order (true parallelism).
- $\circ (T_{\text{body}}, T_{\text{redo}})$: execute the body, then optionally the redo branch, repeating until the loop exits.

Process trees always generate **sound** models, avoiding problems like deadlocks, live-locks, dead transitions, and unreachable states. A **deadlock** occurs when no transition is enabled but the final marking has not been reached; a **livelock** occurs when transitions fire indefinitely without reaching the final marking; a **dead transition** cannot fire in any reachable marking; and an **unreachable state** is a marking no firing sequence from the initial marking can produce. Because the Inductive Miner composes sound building blocks, the resulting model is guaranteed to be free of all these problems. Every process tree can be automatically converted into a Petri net, so the two representations are interchangeable.

The algorithm discovers a process tree using a divide-and-conquer strategy. Starting with the full log, it tries to detect a **cut** — a partition of activities into groups reflecting one of the four operators:

- **Sequence cut**: activities can be grouped and ordered so that directly-follows relations only go forward, never in reverse.
- **Exclusive choice cut**: groups are mutually exclusive and each case belongs to exactly one group.
- **Parallel cut**: each group can follow or precede any other, indicating free interleaving.
- **Loop cut**: one group forms the loop body and another forms the redo branch.

If no cut is found, a fall-through strategy is applied. The algorithm recurses on each sublog until all sublogs contain a single activity.

The **Inductive Miner Infrequent (IMf)** variant — used in this thesis via PM4Py — filters out infrequent directly-follows pairs before detecting cuts. This makes the algorithm more robust to noise in large real-world logs, at the cost of some fitness on rare variants.

Example. Consider $L = [\langle A, B, D \rangle^4, \langle A, C, D \rangle^3]$. The algorithm first checks for a sequence cut: A is always first, D is always last, and nothing in $\{B, C\}$ directly follows D or is directly followed by A . A sequence cut with three groups $A_1 = \{A\}$, $A_2 = \{B, C\}$, $A_3 = \{D\}$ is confirmed. The algorithm recurses on each:

- $A_1 = \{A\}$: single activity — leaf node A .
- $A_2 = \{B, C\}$: the sub-log is $[\langle B \rangle^4, \langle C \rangle^3]$. Since B and C never directly follow each other and no case contains both, an exclusive choice cut applies — leaf nodes B and C under a $\times$ operator.
- $A_3 = \{D\}$: single activity — leaf node D .

The resulting process tree is $\rightarrow (A, \times (B, C), D)$: always start with A , choose exclusively between B and C , then always end with D . This tree is automatically converted into a sound Petri net that correctly reproduces both variants in the log.

Example 1

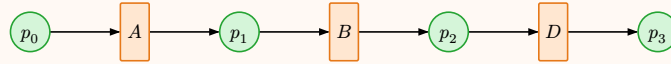
To make these concepts concrete, consider the simple sequential process $A \rightarrow B \rightarrow D$. The Petri net for this process (shown in 2) has places, transitions, and flow:

$$P = \{p_0, p_1, p_2, p_3\}, \quad T = \{A, B, D\}, \quad (11)$$

$$F = \{(p_0, A), (A, p_1), (p_1, B), (B, p_2), (p_2, D), (D, p_3)\}. \quad (12)$$

The initial marking is $M_0 = \{p_0 : 1\}$ and the final marking is $M_f = \{p_3 : 1\}$.

Figure 2: The Petri net for the sequential process $A \rightarrow B \rightarrow D$.



Source: Own elaboration.

To verify that $\langle A, B, D \rangle$ is accepted:

- Step 1: A is enabled (token in p_0). Fire A : $M_1 = \{p_1 : 1\}$.
- Step 2: B is enabled (token in p_1). Fire B : $M_2 = \{p_2 : 1\}$.
- Step 3: D is enabled (token in p_2). Fire D : $M_3 = \{p_3 : 1\} = M_f$. Accepted.

Now consider $\langle A, D \rangle$ (skipping B). After A fires, the token is in p_1 . Transition D requires a token in p_2 , which is empty, so D cannot fire. This correctly reflects that B cannot be skipped in this model.

In process mining, transitions correspond to activities, places represent states between activities, and tokens represent the current execution state of a process instance. Petri nets support both visual interpretation and formal analysis of properties such as reachability and soundness, and PM4Py uses them as the primary model format for conformance checking and performance analysis.

2.4 Conformance checking

Given both a process model and an event log, it is possible to compare real-life process executions with the intended process design. This comparison — known as **confor-**

mance checking — is a core application of process mining, helping organizations identify rule violations, unexpected deviations, and potential data quality problems. There are two main approaches.

Token-based replay simulates each trace from the log on the Petri net by firing transitions in the observed order. Each time a transition fires, it **consumes** one token from each input place and **produces** one token in each output place. If the trace follows a valid path, the replay proceeds without interruption. If a required token is missing, an artificial token is added (**missing token**); if tokens remain in non-sink places at the end, they are **remaining tokens**. The fitness of a single trace is:

$$\text{fitness}(\sigma) = \frac{1}{2} \left(1 - \frac{p}{c} \right) + \frac{1}{2} \left(1 - \frac{r}{q} \right), \quad (13)$$

where p is the number of missing tokens, c is the total tokens consumed, r is the number of remaining tokens, and q is the total tokens produced. The ratio p/c normalizes missing tokens by total consumption, so a single missing token in a long trace is penalized less than in a short one; r/q does the same for remaining tokens. A fitness of 1 indicates a perfectly fitting trace; lower values indicate greater deviation. Overall log fitness is the weighted average across all cases (Aalst, 2022).

Example 2

Continuing from 1, consider replaying $\sigma = \langle A, D \rangle$ on the net in 2 (B is skipped). After A fires, the marking is $\{p_1 : 1\}$. Transition D requires a token in p_2 , which is missing, so one artificial token is added: $p = 1$, and the marking becomes $\{p_1 : 1, p_2 : 1\}$. Firing D yields $\{p_1 : 1, p_3 : 1\}$. Place p_1 still holds a token that was never consumed because B was not fired: $r = 1$. With $c = 2$ and $q = 2$:

$$\text{fitness} = \frac{1}{2} \left(1 - \frac{1}{2} \right) + \frac{1}{2} \left(1 - \frac{1}{2} \right) = \frac{1}{2}. \quad (14)$$

This reflects the significant deviation: one activity was skipped and one token was left unconsumed.

Alignment-based conformance finds the closest valid model execution for each observed trace. Each **alignment step** is a pair (l, m) , where l is either an observed activity or the skip symbol $\gg$, and m is either a model activity or $\gg$. Three types of steps are possible:

- **Synchronous step** (a, a) : activity a occurs in both log and model — no deviation.
- **Log step** $(a, \gg)$: activity a occurs in the log but is not expected by the model — unexpected behavior.
- **Model step** $(\gg, a)$: activity a is expected by the model but absent in the log — skipped behavior.

The cost of an alignment is the total number of non-synchronous steps. The **optimal alignment** minimizes this cost and is typically found using the A^* algorithm on an alignment state space.

Example 3

Continuing from 1 and 2, the optimal alignment for $\langle A, D \rangle$ against the model accepting $\langle A, B, D \rangle$ is:

Table 3: Optimal alignment for $\langle A, D \rangle$ against the model $\langle A, B, D \rangle$.

Log step	Model step	Type
A	A	Synchronous — no deviation
$\gg$	B	Model step — B expected but not observed
D	D	Synchronous — no deviation

Source: Own elaboration.

The alignment cost is 1, identifying precisely that the only deviation is the absence of B . This is more informative than the fitness score of 0.5 alone.

Alignment-based conformance provides more precise diagnostics than token replay but is also more computationally intensive, especially for long traces or large models. In practice, token replay is used for quick fitness estimates, while alignment-based conformance is reserved for detailed per-trace analysis.

Deviations found through conformance checking can reflect data quality issues, exceptional cases, policy violations, or genuine changes in the process over time. Understanding the nature and frequency of these deviations is a key outcome of conformance analysis.

2.5 Performance analysis and process intelligence

Performance analysis adds a time dimension to the structural insights gained through process discovery and conformance checking. Because event logs record a timestamp for each event, it is possible to measure the elapsed time between activities and the overall throughput time for each case. When lifecycle attributes are available — such as explicit start and complete events for each activity — the service time of individual activities can also be estimated. Projecting these measurements onto the process model produces a **performance-annotated** view that highlights which parts of the process consume the most time.

It is important to note that event logs typically capture the time **between** recorded events, not the exact duration of each activity. If event e_i is recorded at time t_i and the next event e_{i+1} at time t_{i+1} , the elapsed time $t_{i+1} - t_i$ includes both waiting time and processing time, which cannot be separated from the log alone.

The **throughput time** of a case c is the elapsed time between the timestamps of its first and last recorded events:

$$TT(c) = \max_{e \in c} t(e) - \min_{e \in c} t(e), \quad (15)$$

where $t(e)$ is the timestamp of event e . This does not account for any time before the first recorded event or after the last, so it may underestimate the true end-to-end duration if the log is incomplete.

The **waiting time** between two consecutive events e_i and e_{i+1} in a trace is $t(e_{i+1}) - t(e_i)$. The **service time** of an activity — the time required to actually perform it — can only be measured when the log records both a start and a complete lifecycle event for that activity instance.

Several summary statistics are routinely reported to characterize throughput time across all cases:

Table 4: Common performance indicators for throughput time analysis.

KPI	Meaning
Mean throughput time	Average case duration across all cases
Median throughput time	Middle value — robust to extreme outliers
P95 throughput time	95% of cases complete within this time
Minimum	Fastest observed case duration
Maximum	Slowest observed case duration
Coefficient of variation	Standard deviation divided by mean — measures relative variability

Source: Own elaboration.

A low coefficient of variation indicates that most cases take approximately the same time; a high value signals greater variability, possibly caused by bottlenecks, resource constraints, or exceptional cases.

When elapsed times are projected onto the process model — labeling each arc with the median or mean duration computed from all cases passing through it — a **performance-annotated model** is obtained. In formal performance models, each transition is associated with a probability distribution for its duration, such as an exponential distribution with a given rate, enabling simulation and more detailed analytical modeling. This distributional view underlies the simulation models demonstrated in 4.

Process intelligence builds on these ideas by linking process data with forecasting, simulation, and optimization. Once a performance baseline has been established, it becomes possible to predict outcomes for cases in progress, test proposed process changes through simulation, or build real-time dashboards that alert managers when a case is projected to exceed a target throughput time. The mathematical foundations introduced in this chapter therefore remain relevant across the full scope of process intelligence, not only for the basic retrospective tasks of discovery and conformance checking.

2.6 Event data representation in Python

From an implementation perspective, event data in Python is typically managed as either tabular data structures or specialized event-log objects. Following the XES stan-

dard, the basic tabular format uses four core columns — case identifier, event identifier, activity name, and timestamp — plus any additional attributes such as resource, cost, or organizational unit. This structure is straightforward to work with using the `pandas` library and its `DataFrame`, which is standard across the Python data science ecosystem.

Process mining libraries convert these tables into richer event-log objects that preserve trace structure and support discovery algorithms. In PM4Py, the conversion is performed by `pm4py.format_dataframe`, which requires specifying the columns for case identifier, activity name, and timestamp, followed by `pm4py.convert_to_event_log` to produce the internal log object. This pipeline is demonstrated in the synthetic introductory example in 4, which transforms a plain `DataFrame` into a format suitable for the Inductive Miner.¹

A more advanced representation is the **Object-Centric Event Log (OCEL)**, version 2.0.² Unlike classical event logs, which link each event to a single case, OCEL allows a single event to be associated with multiple objects of different types simultaneously. For example, a procurement event may relate simultaneously to a purchase order, an order item, a vendor, and a resource. PM4Py reads OCEL files in JSON format using `pm4py.read_ocel_json`, producing an object with separate `DataFrames` for events and objects, along with metadata about object types and event attributes.

Flattening converts an OCEL into a classical event log by selecting one object type as the case identifier. In PM4Py, `pm4py.ocel_flattening(ocel, object_type)` performs this step. When one event is related to multiple objects of the selected type, it appears in multiple cases in the flattened log, inflating the case count; when the selected type is not linked to all events, some events may be lost. Managing these artifacts is an important preprocessing step before applying classical discovery algorithms to object-centric data.

In the practical examples in 4, both representations are used. The synthetic introductory example uses the tabular format to illustrate the basic conversion pipeline. The BPI Challenge 2019 case study uses the OCEL representation, and the flattening step is demonstrated in detail, showing how object-centric data must be preprocessed before classical discovery algorithms can be applied.

2.7 Exploratory description of the BPI Challenge 2019 dataset

The primary real-world case study in this thesis is based on the BPI Challenge 2019 dataset³, provided in OCEL format. Originating from a real procurement process at a large multinational company, the dataset was released as part of the annual BPI

¹More information about the XES standard can be found at <https://www.tf-pm.org/resources/xes-standard/>.

²More information about the OCEL standard can be found at <https://www.ocel-standard.org/>.

³The dataset is publicly available at: https://data.4tu.nl/articles/dataset/BPI_Challenge_2019/12715853/1.

Challenge competition⁴ linked to the International Conference on Process Mining⁵. It is a well-established benchmark in academic research and serves here as a representative example of a large, complex, real-world event log.

The dataset contains approximately 1,595,923 events and 330,685 objects distributed across four object types: purchase orders (PO), purchase order items (POItem), resources (Resource), and vendors (Vendor). Each event is timestamped and labeled with an activity name, and carries rich business context — including company code, document type, item category, spending area, and vendor details. This wealth of attributes makes the dataset useful not only for control-flow analysis but also for performance analysis and organizational mining.

Table 5: Overview of the BPI Challenge 2019 dataset.

Characteristic	Value
Total events	1,595,923
Total objects	330,685
Object types	4 (PO, POItem, Resource, Vendor)
Format	OCEL (JSON)
Domain	Procurement

Source: Own elaboration based on the BPI Challenge 2019 dataset.

To make the data suitable for classical process mining, the OCEL log is **flattened** with respect to the POItem object type — selected because it is the most frequent type in the log. Each unique POItem becomes one case in the resulting classical event log. In the sample used for the practical examples in 4, the flattened log contains 43,898 cases (after filtering out records with timestamps before 2018, which reflect data entry errors in the ERP system).

Table 6: Throughput time statistics for the BPI Challenge 2019 dataset (flattened by POItem).

Metric	Value
Cases with at least 2 events	248,899
Mean throughput time	72.3 days
Median throughput time	64.3 days
95th percentile	143.0 days
Minimum	0.0 days
Maximum	25,670.6 days

Source: Own elaboration based on the BPI Challenge 2019 dataset.

On average, processing a purchase order item takes approximately two months from the first to the last recorded event. The 95th percentile of 143 days shows that a meaningful minority of cases take significantly longer, and rare outliers — possibly

⁴The BPI Challenge 2019 competition page: <https://icpmconference.org/2019/icpm-2019/contests-challenges/bpi-challenge-2019/>.

⁵The ICPM conference series: <https://icpmconference.org/>.

open or anomalous cases — extend to over 25,000 days. This right-skewed distribution is typical of procurement processes, where the majority of cases complete within a predictable range but a small number of exceptions take considerably longer.

These statistics highlight several important characteristics of the dataset. First, its volume demands efficient tools and careful preprocessing before any analysis can begin. Second, the presence of four object types means that a single-case model would lose important relational information about the process. Third, the combination of control-flow data with rich business attributes enables a broad range of process mining tasks beyond simple discovery.

In 4, the dataset is loaded using `pm4py.read_ocel_json`, flattened to a case-centric format, and sampled to a manageable size before discovery algorithms are applied. All the mathematical foundations introduced in this chapter — traces, variants, Petri nets, and throughput time — are applied directly, demonstrating how theory connects to concrete analytical results.

Chapter 3

The tools available in Python

This chapter surveys the main Python libraries available for process mining and process intelligence, organized into five categories: core process mining libraries, predictive and AI-oriented tools, simulation and optimization tools, visualization libraries, and complementary data processing packages. PM4Py is identified as the most comprehensive and actively maintained library, supporting the full analytical workflow. The chapter concludes with a comparative summary table.

The Python ecosystem for process mining and process intelligence has expanded considerably over the past decade. It now supports a wide range of activities: loading and exploring event logs, discovering process models, checking conformance, predicting future outcomes, and simulating process changes. This chapter provides a structured overview of the main Python tools available for these purposes, organized into five categories: core process mining libraries, predictive and AI-oriented tools, simulation and optimization tools, visualization libraries, and complementary data processing packages. For each tool, the discussion covers its primary purpose, supported data formats, key algorithms, and practical strengths and limitations. The chapter concludes with a comparative summary table.

The tools discussed here were selected based on a review of academic publications from the International Conference on Process Mining (ICPM) and related venues, the Python Package Index (PyPI), and active GitHub repositories related to process mining. The focus is on tools that are actively maintained, widely cited in the literature, or representative of an important category of functionality. Commercial solutions are included only where they provide a Python interface accessible to practitioners, as in the case of PyCelonis.

3.1 Core process mining libraries

The foundation of the Python process mining ecosystem is its core libraries, which implement the fundamental workflow: reading event data, exploring log statistics, discovering process models, checking conformance, and analyzing performance. This section describes the four main libraries in this category: PM4Py, PM4Py-Streaming, Simod, and PMLab.

3.1.1 PM4Py

PM4Py is the central library in the Python process mining ecosystem. Developed and maintained by the Process Intelligence Solutions group (with academic roots at Fraunhofer FIT and RWTH Aachen University), PM4Py was first released in 2018 and has since become the de facto standard for process mining in Python, with frequent updates and a growing user base in both academia and industry.

PM4Py covers the full process mining workflow. Its main capabilities include:

- **Event log input and output:** PM4Py reads and writes event logs in multiple formats, including XES (the IEEE standard), CSV files via pandas DataFrames, and OCEL JSON files for object-centric logs. Reading a standard XES file requires only a call to `pm4py.read_xes()`, while `pm4py.read_ocel_json()` handles OCEL logs. Both functions return structured objects compatible with PM4Py’s full suite of analytical tools.
- **Log statistics and exploration:** PM4Py computes basic statistics such as activity, case, and event counts; variant frequencies; start and end activities; and directly-follows relations. Functions such as `pm4py.get_variants()` and `pm4py.discover_dfg()` operate on both classical event logs and OCEL structures.
- **Process discovery:** Several discovery algorithms are implemented. The Alpha algorithm (`pm4py.discover_petri_net_alpha()`) builds a Petri net from directly-follows relations. The Inductive Miner (`pm4py.discover_petri_net_inductive()`) produces a sound Petri net via a process tree and is the main algorithm used in the case study in 4. The Heuristics Miner (`pm4py.discover_petri_net_heuristics()`) applies frequency thresholds to filter noise before model construction, making it more robust to infrequent behavior than the Alpha algorithm. All discovery functions accept both classical event-log objects and pandas DataFrames with specified column names.
- **Conformance checking:** PM4Py provides both token-based replay (`pm4py.fitness_token_based_replay()`) — fast and suitable for large logs — and alignment-based conformance (`pm4py.fitness_alignments()`), which yields detailed per-trace diagnostics at higher computational cost. Both return a dictionary of fitness metrics including the overall fitness score.
- **Performance analysis:** PM4Py calculates throughput times, activity durations, and waiting times for timestamped event logs. These measures can be projected onto a discovered process model to produce performance-annotated visualizations that highlight bottlenecks.
- **Object-centric process mining:** PM4Py is currently the only Python library with comprehensive native support for OCEL. It provides `pm4py.ocel_flattening()` to convert an OCEL log to a classical format, and `pm4py.discover_ocdfg()` to compute object-centric directly-follows graphs without flattening.

Strengths and limitations: PM4Py’s main strengths are its broad coverage, active development, thorough documentation, and robust OCEL support. It is the only Python library offering the complete process mining workflow in a single package. Its main limitations are computational: alignment-based conformance and some discovery algorithms can be slow on very large logs, and advanced features such as simulation are not currently in scope.

3.1.2 PM4Py-Streaming

PM4Py-Streaming is an extension of the core PM4Py library, designed specifically for situations where event data arrives continuously — as a stream — rather than in a complete, stored log. This is particularly useful in real-world scenarios like live monitoring of ERP systems or logistics platforms, where new events are generated in real time. In these cases, analysts need to keep the process model up to date without having to reprocess all historical events each time something new happens.

The library implements online versions of key process mining algorithms. These algorithms update their internal state incrementally with each incoming event, without storing the full event history. For example, the online directly-follows graph keeps running frequency counts for each activity pair and updates them instantly as new events come in. There is even an online variant of the Inductive Miner, which can adapt the discovered process tree as stream statistics change over time. PM4Py-Streaming is designed to work seamlessly with PM4Py’s standard data structures, so any models discovered from the stream can be further analyzed using the conformance and performance functions in PM4Py.

Strengths and limitations: PM4Py-Streaming is currently the only Python library dedicated to online process mining. Its main limitation is that it supports fewer algorithms than the standard batch PM4Py library, and the approximations required for online discovery can sometimes reduce model quality compared to what is possible with a complete, static log.

3.1.3 Simod

Simod is a Python tool focused on simulation-based optimization of business processes. It takes a BPMN process model — usually one produced by process discovery — as input, estimates simulation parameters from an event log, and then generates an optimized simulation configuration. The ultimate goal is to identify the combination of resource assignments, activity durations, and scheduling rules that will minimize some target objective, such as overall throughput time or cost.

Simod’s workflow proceeds in three main phases. First, it extracts and converts a process model from the event log to BPMN format. Next, it estimates simulation parameters — such as resource pools, activity duration distributions, and arrival rates — from the log data. Finally, a Bayesian optimization loop runs multiple simulation experiments with different parameter settings, searching for the configuration that best achieves the chosen objective.

Strengths and limitations: Simod fills an important gap in the Python ecosystem by linking process mining with simulation-based optimization. However, it requires a fairly clean and well-structured BPMN model as input, so it is most useful after a successful process discovery step has already been completed. It also depends on tools like Graphviz and a BPMN simulation engine, which can add to the installation and setup complexity.

3.1.4 PMLab

PMLab is a research-oriented toolkit for process mining in Python. It provides implementations of a variety of process discovery and conformance algorithms, including several that are not available in PM4Py. PMLab is aimed mainly at researchers who want to experiment with or develop new process mining algorithms, rather than practitioners who need a polished, production-ready tool.

The library supports classical event logs in XES format and offers Python implementations of the Alpha algorithm, several variants of the Heuristics Miner, and various conformance checking methods. PMLab is designed to be modular and extensible, making it easier for researchers to add or modify algorithms without having to overhaul the entire library.

Assessment: PMLab is a valuable resource for researchers who need to dive deep into algorithm development and modification. However, it is less suited for production use — development has slowed, documentation is limited, and it lacks support for newer features like OCEL and the Inductive Miner Infrequent variant.

3.1.5 BupaR (Python interface)

BupaR is a comprehensive process mining framework originally developed for the R programming language (Janssenswillen *et al.*, 2019). It introduces a structured grammar for process analysis — a set of consistent operations for filtering, aggregating, and visualizing event data — centered around a dedicated event log object. This approach ensures that all analyses are built on a consistent data structure. BupaR’s expressive and readable pipeline syntax has influenced several tools in the Python ecosystem and remains popular in academic research for its clarity and reproducibility.

Python users can access BupaR’s functionality through the rpy2 bridge, which allows R functions to be called directly from within a Python environment. This means analysts can leverage BupaR’s powerful event log manipulation and visualization tools — such as dotted charts and process maps — without leaving Python. These visualizations are widely seen in academic papers as clear, succinct summaries of process behavior.

Strengths and limitations: BupaR’s greatest strength is its expressive, consistent grammar for process analysis, which supports a broader range of log manipulation and visualization operations than PM4Py’s current Python API. Its main limitation for Python users is the need for a working R installation and the rpy2 bridge, which adds a layer of complexity compared to pure-Python solutions.

3.2 Predictive and API-oriented tools

A rapidly growing set of Python tools connects process mining with machine learning and artificial intelligence, enabling predictive process monitoring, automated decision support, and intelligent process automation. The following are some of the most prominent libraries in this area.

3.2.1 ML4ProM

ML4ProM is a Python framework that brings together process mining techniques and machine learning models to support predictive process monitoring. This area focuses on forecasting future states for running process instances — for example, predicting whether a case will finish on time, what the next activity will be, or how long a case is likely to take.

The ML4ProM workflow follows a standard pattern: first, the event log is preprocessed to extract features from partial traces. These features are then used to train a machine learning model, such as a classifier or regressor from scikit-learn. The trained model can then be applied to new, ongoing cases to generate predictions. ML4ProM provides several feature extraction strategies that are commonly used in process mining, including sequence encoding (representing the activity prefix as a fixed-length vector), last-state encoding (capturing the most recent activity and its attributes), and aggregate encoding (using statistics over the trace prefix).

Choosing the right feature encoding is important, as it determines how much of the process history is captured and how well the features represent the process state. ML4ProM enables analysts to experiment with different encodings and compare their predictive accuracy using standard metrics such as accuracy, AUC-ROC, and mean absolute error.

Strengths and limitations: ML4ProM’s main strength is its seamless integration of process mining feature extraction with scikit-learn-compatible models. Its main drawbacks are the need for a reasonably large labeled training dataset and the current lack of support for object-centric event logs.

3.2.2 PM4Py-ML

PM4Py-ML is an extension of PM4Py that directly integrates machine learning pipelines with PM4Py event-log objects and analysis functions. It provides scikit-learn-compatible transformers to convert PM4Py event logs into feature matrices, making it straightforward to train machine learning models for predictive process monitoring within the PM4Py ecosystem.

The library supports several feature extraction methods that work with PM4Py’s internal log format, including counts of activity occurrences, frequencies of directly-follows pairs, and time-based features such as remaining and elapsed time. These features can be passed to any scikit-learn estimator using standard pipeline objects. The `LogTransformer` class converts a PM4Py event log directly into a NumPy feature matrix, which can then be supplied to any scikit-learn classifier or regressor.

Strengths and limitations: PM4Py-ML’s main advantage is its tight integration with PM4Py, which streamlines the transition from process mining analysis to machine learning modeling. Its main limitations are similar to ML4ProM: it requires labeled training data and does not natively support object-centric logs.

3.2.3 PyCelonis

PyCelonis is the official Python API for the Celonis Process Intelligence platform — a leading commercial solution used by large organizations for process monitoring, compliance, and improvement at scale. With PyCelonis, analysts can interact with the Celonis platform directly from Python, enabling automated analyses, extraction of data and results, and integration of Celonis insights into broader data science workflows.

Key features of PyCelonis include connecting to a Celonis instance using an API key for authentication, extracting event log data using PQL (Process Query Language), sending analysis results and action recommendations back to the platform, and integrating Celonis with robotic process automation (RPA) tools for automated process execution.

Strengths and limitations: PyCelonis’s primary strength is its ability to bridge Celonis’s commercial process intelligence infrastructure with the flexibility of Python’s data science ecosystem. The main limitation is that it is tightly tied to the Celonis platform, which requires a commercial subscription and is therefore not accessible for academic or small-scale projects without an institutional license.

3.2.4 BPMN-Python and workflow automation

The BPMN-python library and related tools enable users to extract, manipulate, and visualize BPMN process models in Python. BPMN (Business Process Model and Notation) is the industry-standard graphical language for representing business processes, widely adopted in workflow automation systems. PM4Py can export discovered Petri nets to BPMN format, after which bpmn-python can import these models, modify them programmatically, or render them as interactive visualizations. This creates a practical bridge between process mining analysis and operational business process management systems.

Strengths and limitations: BPMN-python and similar tools are most valuable in organizations that already use BPMN-based workflow systems. For purely analytical purposes, they may add unnecessary complexity compared to the Petri net and process tree representations already supported by PM4Py.

3.2.5 Predictive Process Monitoring Toolkit (PPMT)

The Predictive Process Monitoring Toolkit (PPMT) is a Python implementation of the benchmark framework for predictive process monitoring research described by Teinmaa et al. (Teinmaa *et al.*, 2019). It supports the full experimental pipeline: reading XES event logs, extracting prefixes of various lengths, applying multiple feature encoding strategies (such as index, sequence, and aggregate encoding), training a range of machine learning models (including logistic regression, random forest, and XGBoost), and evaluating predictive performance using metrics such as AUC-ROC and mean absolute error.

PPMT is primarily a research tool, designed to reproduce and extend the benchmark results of Teinmaa et al. Its main value is that it allows researchers to compare new

predictive methods against a well-established baseline using a consistent experimental protocol — without having to build the evaluation infrastructure from scratch.

Strengths and limitations: PPMT’s main strength is its comprehensive benchmark coverage: it enables researchers to reproduce published results and incorporate new methods for standardized comparison. Its limitations are that it is not designed for real-time deployment, documentation is limited beyond the associated research paper, and it is not actively maintained as a production system.

3.3 Simulation and optimization tools

Simulation and optimization tools extend the reach of process mining from understanding past behavior to designing and testing improvements for future processes. The following section introduces the main Python tools for simulation and optimization: SimPy, BPSim, and ProcessOptimizer.

3.3.1 SimPy

SimPy is a widely used Python library for discrete-event simulation. In this approach, a system is modeled as a sequence of events that occur at specific points in time, each capable of changing the system state and scheduling future events. This fits business processes well, since activities are performed by resources over time and cases enter and exit the process according to statistical patterns.

Although SimPy was not developed specifically for process mining, it integrates well with process mining results. The typical workflow begins with process mining: the event log is analyzed to discover the process structure and estimate activity durations. These parameters are then used to configure a SimPy simulation, defining process generators, resource pools, and arrival rates. Running the simulation generates synthetic event data under various scenarios, which can then be analyzed further using PM4Py.

For example, modeling a sequential process with three activities in SimPy involves defining a function that requests a resource, waits for a service time (commonly modeled as an exponential distribution), and releases the resource after each activity. By adjusting the number of available resources and running the simulation, it is possible to estimate how staffing changes might affect throughput time — supporting informed decisions before any change is made to the actual process.

Assessment: SimPy is simple, well-documented, and widely adopted, making it practical for this kind of analysis. Its main limitation is integration: there is currently no automatic way to convert a process model discovered with PM4Py into a SimPy simulation. For simple models, building this bridge manually is straightforward, but as model complexity increases the effort grows considerably.

3.3.2 BPSim

BPSim is a simulation standard designed for BPMN process models. It defines a structured, XML-based format for specifying simulation parameters — such as resource pools, activity duration distributions, and routing probabilities — directly within a

BPMN model file. Tools that support BPSim, including Simod, can read these parameters and run structured simulation experiments without requiring custom code. BPSim is particularly useful when a process model is already available in BPMN format and the goal is to systematically evaluate multiple alternative configurations.

3.3.3 Process Optimizer

Process Optimizer is a Python library for Bayesian optimization, developed by Novo Nordisk Research. Built on top of scikit-optimize, it provides a framework for tuning process parameters in situations where each evaluation is costly — such as when running a detailed simulation.

In the context of process mining, Process Optimizer can be used to find simulation parameter settings that minimize throughput time or maximize resource utilization. The analyst defines an objective function — taking a set of parameters, running the simulation, and returning a performance score — and Process Optimizer suggests new settings, evaluates them, and builds a probabilistic model of the objective function to focus subsequent evaluations on the most promising regions of the parameter space.

Strengths and limitations: Process Optimizer is efficient: it identifies good parameter settings without exhaustively evaluating every possible combination, which is especially valuable when simulation runs are time-consuming. Its main limitation is that an appropriate objective function and simulation model must be defined in advance, so it is most useful after the process structure has already been established through discovery.

3.4 Visualization libraries

Visualization is central to process analysis, both for exploring event data and for communicating results to stakeholders. The Python process mining ecosystem includes several libraries supporting different types of process visualizations.

3.4.1 Graphviz

Graphviz is an open-source graph visualization tool and serves as the primary rendering engine for process models in PM4Py. When a model — such as a Petri net, process tree, or directly-follows graph — is discovered, PM4Py automatically converts it to Graphviz’s DOT format and calls the `dot` command-line tool to produce images. No DOT code needs to be written manually. The resulting images can be saved as PNG or PDF files for reports, or displayed interactively in Jupyter notebooks.

In this thesis, Graphviz is used to visualize the Petri net models discovered from the BPI Challenge 2019 dataset. If Graphviz is not installed, PM4Py falls back on a built-in renderer for directly-follows graphs, but full Petri net visualization requires Graphviz to be present in the system PATH.

Strengths and limitations: Graphviz produces high-quality, publication-ready visualizations. Its main drawback is that it must be installed separately from PM4Py, which can occasionally cause installation or compatibility issues on some systems.

3.4.2 Plotly and Dash

Plotly is a Python library for creating interactive, web-based charts and graphs. Unlike static plotting libraries such as Matplotlib, Plotly enables interactivity — zooming, panning, filtering, and hovering for additional detail. Dash, built on top of Plotly, is a framework for building full-featured interactive dashboards that combine visualizations, tables, and user controls.

For process mining, Plotly and Dash are particularly useful for visualizing performance metrics interactively. For example, `plotly.express.histogram()` can display the distribution of throughput times, while `plotly.express.bar()` can compare variant frequencies over time. Dash enables dashboards that update dynamically as filters — such as date range or document type — are adjusted.

Strengths and limitations: Plotly and Dash are the most capable Python tools for building interactive dashboards for process performance analysis. Their main limitation is that they do not natively support rendering process models such as Petri nets or directly-follows graphs — for those, Graphviz remains the standard tool.

3.4.3 NetworkX

NetworkX is a library for creating, manipulating, and analyzing complex networks and graphs. In the context of process mining, it is used to analyze the structure of directly-follows graphs — for example, identifying the most central activities, detecting clusters of related activities, or computing path lengths and connectivity. Since PM4Py outputs directly-follows graphs as Python dictionaries, converting them to NetworkX graphs is straightforward. Built-in functions such as `betweenness_centrality()` and `shortest_path()` can then be applied to uncover important structural features of the process.

Strengths and limitations: NetworkX is mature, well-documented, and offers a wide range of graph analysis algorithms. Its main limitation for process mining is that it treats process models as generic directed graphs — it does not understand process mining-specific concepts such as markings or conformance.

3.4.4 Pyvis

Pyvis is a Python library for interactive network visualization, based on the vis.js JavaScript library. With Pyvis, analysts can create interactive browser-based visualizations of directly-follows graphs that can be explored by zooming, panning, and inspecting edge weights. While PM4Py and Graphviz produce static images, Pyvis generates interactive HTML visualizations that can be embedded in Jupyter notebooks or shared as standalone web pages — making them easier to distribute, especially to stakeholders who do not have process mining software installed.

To use Pyvis with PM4Py, the directly-follows graph dictionary is first converted to a NetworkX graph, which is then passed to Pyvis’s `Network` object for rendering. This makes Pyvis a lightweight option when interactivity is more important than publication-quality output.

Strengths and limitations: Pyvis excels at producing interactive, shareable network visualizations with minimal code. However, it does not support Petri net or process tree rendering — only general graph structures — and its visual output is not as polished as Graphviz for formal process model diagrams.

3.5 Complementary data processing libraries

Several general-purpose Python libraries are essential for effective process mining, even though they are not process mining-specific. They provide the underlying infrastructure for data handling, statistical analysis, and machine learning modeling.

3.5.1 Pandas

Pandas is the most important general-purpose Python library for process mining. Its `DataFrame` structure — a two-dimensional table with labeled columns — is used throughout the process mining workflow to represent event logs, compute statistics, and prepare data for analysis.

Most real-world event log data begins as CSV files, Excel spreadsheets, or database query results. Pandas provides the tools needed to read these formats, clean and transform data, filter cases or events, and compute derived attributes such as throughput times. PM4Py integrates directly with pandas `DataFrames`: `pm4py.format_dataframe()` specifies column roles, and `pm4py.convert_to_event_log()` creates the internal event log object. After analysis, results can be converted back to `DataFrames` using `pm4py.convert_to_dataframe()` for further manipulation with standard pandas operations.

3.5.2 NumPy

NumPy is the numerical foundation of Python’s data science stack, including pandas and scikit-learn. In process mining, it operates mostly behind the scenes as a dependency, though analysts may use it directly for custom calculations or numerical methods not available in higher-level libraries.

3.5.3 Scikit-Learn

Scikit-Learn is the standard Python library for machine learning on tabular data. It provides a wide range of algorithms for classification, regression, and clustering, together with tools for model evaluation, cross-validation, and pipeline construction.

In process mining, Scikit-Learn is used to build predictive models on features extracted from event logs. The typical workflow involves extracting a feature matrix using tools such as PM4Py-ML or ML4ProM, splitting the data into training and test sets, training a model, and evaluating its performance. The `Pipeline` class is especially useful for chaining data preparation and modeling steps into a single, reusable object.

3.5.4 XGBoost and LightGBM

XGBoost and LightGBM are high-performance gradient boosting libraries that deliver strong results on tabular machine learning tasks. They combine many weak decision

tree models into a powerful predictor using gradient descent. In process mining, they are commonly used for tasks such as remaining time prediction and outcome classification, and both are compatible with scikit-learn pipelines.

3.5.5 TensorFlow and PyTorch

TensorFlow and PyTorch are the leading deep learning frameworks in Python. They are used in advanced process mining research for sequence modeling tasks such as next-activity prediction, remaining time estimation, and anomaly detection. Techniques including recurrent neural networks (RNNs), long short-term memory networks (LSTMs), and Transformer-based models are commonly applied to event sequences.

While deep learning has gained traction in process mining research since around 2016, these approaches typically require substantially more training data and computational resources, and the resulting models are generally less interpretable. For most practical applications, gradient boosting methods offer a better balance between predictive accuracy, interpretability, and computational efficiency.

3.6 Academic and community resources

In addition to software tools, several academic and community organizations play a vital role in the process mining ecosystem.

The **IEEE Task Force on Process Mining** (tf-pm.org) is the main academic body for the field. It is responsible for setting the scope, principles, and standards of process mining, and published the landmark Process Mining Manifesto (Aalst and others, 2012), which outlines key tasks, challenges, and guiding principles.

The **International Conference on Process Mining** (ICPM, icpmconference.org) is the leading conference for researchers in the field. Held annually, ICPM also hosts the BPI Challenge — an open, competitive event where researchers analyze publicly available event logs. The BPI Challenge datasets, including the 2019 edition used in this thesis, have become standard benchmarks in process mining research (Waal *et al.*, 2019).

The **Fraunhofer FIT Process Intelligence Group** (fit.fraunhofer.de) is the research institute behind PM4Py and related tools. The group oversees PM4Py’s ongoing development, maintains its documentation, and conducts research on new algorithms and applications. For practitioners, Fraunhofer FIT is the main point of contact for support beyond the official documentation.

3.7 Summary

The following table provides an overview of the Python tools discussed in this chapter, grouped by category, with notes on their primary use and practical maturity.

Table 7: Overview of Python tools for process mining and process intelligence.

Category	Tool	Primary use	Maturity
Core PM	PM4Py	Discovery, conformance, performance, OCEL	High
	PM4Py-Streaming	Online / real-time process mining	Medium
	Simod	Simulation-based process optimization	Medium
	PMLab	Research-oriented discovery and conformance	Low
	BupaR	Event log analysis grammar (via rpy2)	Medium
Predictive	ML4ProM	Predictive process monitoring	Medium
	PM4Py-ML	ML pipelines on PM4Py event-log features	Medium
	PyCelonis	Integration with Celonis platform	High
	bpmn-python	BPMN model manipulation and automation	Low
	PPMT	Benchmark framework for predictive monitoring	Low
Simulation	SimPy	Discrete-event simulation	High
	BPSim	BPMN-compatible simulation standard	Medium
	ProcessOptimizer	Bayesian parameter optimization	Medium
Visualization	Graphviz	Process model rendering (Petri nets, DFG)	High
	Plotly / Dash	Interactive dashboards and charts	High
	NetworkX	Graph analysis and custom visualization	High
	Pyvis	Interactive browser-based graph visualization	Medium
Data processing	pandas	Event log manipulation and preprocessing	High
	NumPy	Numerical computation	High
	scikit-learn	Machine learning for process monitoring	High
	XGBoost/LightGBM	Gradient boosting for prediction tasks	High
	TensorFlow/PyTorch	Deep learning for sequence modeling	High

Source: Own elaboration.

Among these, PM4Py stands out as the most comprehensive and actively maintained process mining library in Python. It is the main tool used throughout the practical examples in this thesis, and its ability to support the full process mining workflow — including OCEL — makes it the natural starting point for Python-based process mining.

Other libraries play significant supporting roles: SimPy and Simod enable simulation and optimization; ML4ProM and PM4Py-ML connect process mining with machine learning and predictive monitoring; Plotly, NetworkX, and Graphviz enable effective visualization and communication of results; and general-purpose tools such as pandas and scikit-learn provide the data processing backbone for the entire workflow. Together, this ecosystem is mature enough to support end-to-end process intelligence — from raw event data to predictive models and interactive dashboards — entirely in Python.

Some gaps remain. Python tools for truly object-centric process mining are still limited beyond the basic flattening and OCDFG features in PM4Py. Additionally, there is no widely adopted benchmarking or evaluation framework for comparing process mining tools in Python, making it difficult to assess the relative performance of different algorithms on common datasets.

Chapter 4

Worked examples

This chapter demonstrates how the theoretical concepts from 2 and the Python tools from 3 are applied in practice. Three worked examples are presented: a synthetic introductory example that verifies every step against known results, followed by two progressively more complex analyses of the BPI Challenge 2019 procurement dataset covering process discovery and conformance checking, and variant, throughput, and rework analysis.

This chapter presents three worked examples demonstrating how the theoretical concepts and Python tools from 2 and 3 are applied in practice. The first example uses a small, synthetic event log to make every step of the workflow transparent and verifiable against the formal definitions. The next two examples use the BPI Challenge 2019 dataset, building up the analysis from process discovery and conformance checking to variant analysis, throughput time measurement, and rework detection. Together, these examples demonstrate the core process intelligence workflow — from raw event data to actionable performance insights — using PM4Py as the primary tool.

This progression from synthetic to real-world data is intentional. In the synthetic case, the correct result is known in advance, making it straightforward to verify that every step produces the expected output. In the BPI Challenge 2019 case study, the true underlying model is not known; the goal is to let the tools discover structure from the data and measure process behavior — illustrating both the power and the practical challenges of process mining in a business context.

4.1 Introductory synthetic example

Starting with a synthetic example provides a controlled, well-understood baseline. Unlike real-world datasets — where the true process structure is often unknown or changes over time — a synthetic dataset makes it possible to define exactly the number of cases, the allowed activities, the permitted transitions, and the trace frequencies. This makes it straightforward to verify whether process mining tools produce correct and expected results, matching the theory from 2.

Synthetic examples are especially useful for teaching and learning. The mathematics of Petri nets, alignments, and token replay can seem abstract at first. Working through a small, hand-crafted example — where every trace, directly-follows pair, and marking is easy to inspect — builds the intuition needed for more complex analyses. The synthetic example used here bridges the gap between the formal definitions in 2 and the practical implementations in the following sections.

The example represents a simplified purchasing process with three activities: Create order, Approve order, and Send invoice. In the intended process, every purchase order

follows exactly this sequence, meaning the correct process model is a straightforward three-step linear chain that any competent discovery algorithm should recover. With such a simple structure, all key quantities — traces, variants, directly-follows relations, Petri net structure, fitness scores, and throughput times — can be calculated by hand, making it easy to verify that PM4Py produces the expected results.

The synthetic event log contains nine events across three cases, each following the same activity sequence. The main attributes for each event are shown in 8.

Table 8: Synthetic event log for the introductory example.

Case ID	Activity	Timestamp
1	Create order	2024-01-01 09:00
1	Approve order	2024-01-01 10:00
1	Send invoice	2024-01-01 11:00
2	Create order	2024-01-02 09:15
2	Approve order	2024-01-02 10:30
2	Send invoice	2024-01-02 11:20
3	Create order	2024-01-03 08:45
3	Approve order	2024-01-03 09:50
3	Send invoice	2024-01-03 10:40

Source: Own elaboration.

Each row represents a single recorded event linked to one of three cases. In real-world event logs, many additional attributes are typically present — such as the responsible employee, cost, order amount, or vendor code — but these are omitted here to keep the focus on structure and timing for process discovery and conformance checking.

Mathematical characterization of the log:

Applying the formal definitions from 2, the activity set is:

$$A_L = \{\text{Create order}, \text{Approve order}, \text{Send invoice}\} \quad (16)$$

All three traces are identical:

$$\sigma = \langle \text{Create order}, \text{Approve order}, \text{Send invoice} \rangle \quad (17)$$

The event log is therefore $L = [\sigma^3]$: one unique variant repeated three times. There are three cases ($|L| = 3$), one variant ($|\text{Var}(L)| = 1$), and the average trace length is 3.

Directly-follows relation and graph:

Each trace contains the pairs (Create order, Approve order) and (Approve order, Send invoice). Since all traces are identical, the frequency of each pair is 3. The directly-follows graph has three activity nodes, two directed edges each with weight 3, and a linear path from start to end — confirming that the process is strictly sequential and deterministic.

Process discovery with the Inductive Miner:

Running the Inductive Miner on this log identifies a sequence cut with three groups:

- $G_1 = \{\text{Create order}\}$
- $G_2 = \{\text{Approve order}\}$
- $G_3 = \{\text{Send invoice}\}$

All directly-follows edges move forward with no backward edges. The resulting process tree is:

$$\rightarrow (\text{Create order}, \text{Approve order}, \text{Send invoice}) \quad (18)$$

The corresponding Petri net has four places, three transitions, and a simple linear structure that perfectly matches the intended process.

Python implementation with PM4Py:

The implementation begins by constructing a pandas DataFrame with three columns, using PM4Py-standard column names directly. `pm4py.convert_to_event_log()` groups events by case and sorts them by timestamp, returning an EventLog object with three traces. Process discovery is then performed by calling `pm4py.discover_petri_net_inductive()` on the EventLog, producing a Petri net that faithfully captures the process. The result can be visualized using PM4Py's built-in visualization functions.

```
import pm4py
import pandas as pd
import shutil, os, warnings

warnings.filterwarnings("ignore")

df_synth = pd.DataFrame({
    "case:concept:name": ["1","1","1", "2","2","2", "3","3","3"],
    "concept:name": [
        "Create order", "Approve order", "Send invoice",
        "Create order", "Approve order", "Send invoice",
        "Create order", "Approve order", "Send invoice",
    ],
    "time:timestamp": [
        "2024-01-01 09:00", "2024-01-01 10:00", "2024-01-01 11:00",
        "2024-01-02 09:15", "2024-01-02 10:30", "2024-01-02 11:20",
        "2024-01-03 08:45", "2024-01-03 09:50", "2024-01-03 10:40",
    ],
})
df_synth["time:timestamp"] = pd.to_datetime(df_synth["time:timestamp"])

log_synth = pm4py.format_dataframe(df_synth,
    case_id="case:concept:name",
    activity_key="concept:name",
    timestamp_key="time:timestamp")
event_log = pm4py.convert_to_event_log(log_synth)

print(f"Cases           : {len(event_log)}")
print(f"Total events      : {sum(len(t) for t in event_log)}")

variants = pm4py.get_variants(event_log)
```

```

print(f"Distinct variants: {len(variants)}")

dfg, start_acts, end_acts = pm4py.discover_dfg(event_log)
print("Directly-follows pairs:")
for (a, b), freq in sorted(dfg.items(), key=lambda x: -x[1]):
    print(f" {a} -> {b} (freq: {freq})")

net, im, fm = pm4py.discover_petri_net_inductive(event_log)
print(f"Petri net places      : {len(net.places)}")
print(f"Petri net transitions: {len(net.transitions)}")
print(f"Petri net arcs        : {len(net.arcs)}")

```

Output:

```

Cases          : 3
Total events   : 9
Distinct variants: 1
Directly-follows pairs:
  Create order -> Approve order (freq: 3)
  Approve order -> Send invoice (freq: 3)
Petri net places      : 4
Petri net transitions: 3
Petri net arcs        : 6

```

Performance analysis:

The timestamps in the synthetic log make it possible to compute the throughput time for each case as $TT(c) = \max_{e \in c} t(e) - \min_{e \in c} t(e)$. The results are:

- Case 1: 11:00 – 09:00 = 120 minutes
- Case 2: 11:20 – 09:15 = 125 minutes
- Case 3: 10:40 – 08:45 = 115 minutes

Table 9: Throughput time statistics for the synthetic event log.

Metric	Value
Mean throughput time	120 minutes
Median throughput time	120 minutes
Minimum	115 minutes
Maximum	125 minutes
Standard deviation	approx. 5 minutes
Coefficient of variation	approx. 0.04

Source: Own elaboration.

The coefficient of variation of 0.04 — a very low value — indicates that all three cases complete in nearly identical time frames. This consistency is expected: the process is intentionally designed so that every case follows the same sequence, and the only differences in duration arise from the small variations in start times introduced during dataset construction.

Examining performance within each case reveals more detail about how time is distributed between activities. The waiting time between “Create order” and “Approve order” is 60 minutes in Case 1, 75 minutes in Case 2, and 65 minutes in Case 3, averaging approximately 67 minutes. The waiting time between “Approve order” and “Send invoice” is 60 minutes in Case 1, 50 minutes in Case 2, and 50 minutes in Case 3, averaging approximately 53 minutes. If these waiting times were projected onto the discovered Petri net — annotating each arc with the mean waiting time across all cases — the approval step would appear slightly more time-consuming. In real procurement data, consistently long waiting times before approval could signal a resource bottleneck — for example, if a single manager is responsible for many approvals and cases queue up. Identifying such bottlenecks is one of the most valuable real-world applications of performance-annotated process models.

```
tt = (
    df_synth.groupby("case:concept:name")["time:timestamp"].max()
    - df_synth.groupby("case:concept:name")["time:timestamp"].min()
)
tt_min = tt.dt.total_seconds() / 60

for cid, val in tt_min.items():
    print(f" Case {cid}: {val:.0f} min")
print(f"\nMean      : {tt_min.mean():.1f} min")
print(f"Median     : {tt_min.median():.1f} min")
print(f"CV         : {tt_min.std()/tt_min.mean():.2f}")
```

Output:

```
Case 1: 120 min
Case 2: 125 min
Case 3: 115 min

Mean      : 120.0 min
Median    : 120.0 min
CV        : 0.04
```

Extensions and variations:

The synthetic example here is deliberately minimal, but it can be extended in several ways to deepen understanding before moving to the real-world case study.

Adding a second variant — for example, cases where “Create order” is followed by “Reject order” and then “Close case” — would introduce variability. The directly-follows graph would branch, and the Inductive Miner would detect an exclusive choice, adding a choice operator to the process tree. Conformance checking would still show perfect fitness, as both trace types are accepted. This illustrates how trace diversity drives structural complexity in discovered models.

Introducing a loop — for instance, cases where “Approve order” is followed by “Request revision” and then “Approve order” again — would test the algorithm’s ability to represent rework. The Inductive Miner would identify a loop cut, adding a loop operator to the process tree. Both standard and looping traces would be accepted. This ties into the loop analysis performed later on the BPI Challenge 2019 dataset.

Adding noise — a few traces that violate the intended structure, such as “Send invoice” appearing before “Approve order” — would test the IMf algorithm’s robustness. If these noisy traces are rare, the algorithm filters them out, keeping the model clean and reflecting only the majority behavior. The noisy traces would then stand out in conformance checking, with fitness scores below 1.0 and non-synchronous moves in their alignments. This shows how the IMf variant balances robustness and sensitivity, and how conformance checking helps identify exceptions not covered by the model.

4.2 BPI Challenge 2019: the case study

4.2.1 Data source, format, and structure

The BPI Challenge is an annual competition held in conjunction with the International Conference on Process Mining. Each year, a real organization donates a dataset from one of its operational processes, and participants are invited to analyze it using process mining and related methods. The results are presented at the ICPM workshop, forming a growing body of benchmark analyses that the process mining community uses to evaluate and compare tools and algorithms. These datasets are now some of the most widely cited benchmarks in the field, valued for their complexity and real-world relevance.

The BPI Challenge 2019 dataset comes from a procurement process at a large multinational company. Procurement — the acquisition of goods and services from external suppliers — is a classic domain for process mining due to its economic significance and inherent complexity. A typical procurement process involves multiple stakeholders (purchasing agents, approval managers, finance controllers, suppliers) and spans several systems (ERP, supplier portals, invoice management). With many document types, approval levels, and exception procedures, procurement is highly variable: the same formal process can produce dozens or hundreds of different execution paths, depending on how individual cases are routed. The BPI Challenge 2019 dataset captures this variability in a raw event log collected directly from the company’s systems.

The dataset is provided in OCEL (Object-Centric Event Log) format as a JSON file. As discussed in 2, OCEL was developed to address a key limitation of classical event logs, which require each event to be assigned to exactly one case. In real business processes, however, events often relate to several business objects at once. For example, when an invoice is approved, that event may be linked to the invoice, the purchase order, individual items, the vendor, and possibly a resource or cost center — all at the same time. Classical logs cannot represent these multi-object relationships without losing or duplicating information.

OCEL solves this by allowing each event to be associated with a list of objects from multiple types. The OCEL standard defines a JSON schema with two main arrays: one for events, one for objects. Each event has a unique ID, timestamp, activity label, and an object map specifying which objects of each type are involved. Each object has a unique ID, type, and a dictionary of attributes. PM4Py reads this structure using `pm4py.read_ocel_json()`, which parses the JSON and returns an OCEL object with separate pandas DataFrames for events and objects, plus metadata for object types.

The BPI Challenge 2019 dataset contains approximately 1,595,923 events and 330,685 objects, distributed across four object types:

Table 10: Object types in the BPI Challenge 2019 dataset.

Object type	Approximate count	Description
PO (Purchase Order)	22,000	Document authorizing a purchase from a specific vendor
POItem (Purchase Order Item)	285,000	Individual line item within a purchase order
Resource	650	User, role, or system agent that performed activities
Vendor	23,000	Supplier company from whom goods or services are purchased

Source: Own elaboration based on the BPI Challenge 2019 dataset (Waal *et al.*, 2019).

The POItem type is by far the most common object in the dataset. This is because a single purchase order often includes multiple line items, with each item following its own distinct journey through the procurement process. For example, if a purchase order contains fifty different products, the log will include fifty POItem objects — one for each product. Each line item may go through different approval steps, delivery confirmations, and invoice checks, depending on the product category and the applicable procurement rules. As a result, the POItem level offers the most detailed and information-rich perspective for analyzing the procurement process.

Beyond the timestamp and activity name, each event also includes a range of business context attributes:

Table 11: Business context attributes in the BPI Challenge 2019 event data.

Attribute	Description
cCompany	Company code — identifies the legal entity within the multinational group
cDocType	Document type — classifies the purchase order (standard, subcontracting, etc.)
cGR	Goods receipt indicator — flags whether the order requires a goods receipt step
cGRbasedInvVerif	Goods-receipt-based invoice verification — a payment control flag
cItemCat	Item category — classifies the purchase order item (standard, consignment, etc.)
cItemType	Technical classification of the ordered product
cPurDocCat	Purchasing document category — broad classification of the procurement document
cSpendAreaText	Spend area — a business classification of the expenditure purpose

Source: Own elaboration based on the BPI Challenge 2019 dataset (Waal *et al.*, 2019).

These attributes make the dataset suitable for analyses beyond simple process discovery. Filtering by document type can reveal whether different procurement channels follow

different process patterns. Filtering by spend area can identify whether high-value purchases follow a more rigorous approval procedure than routine ones. Analyzing throughput times broken down by company code can reveal whether different entities within the multinational group process procurement more or less efficiently. While these multi-dimensional analyses are not performed in this thesis, they illustrate the richness of the available data and the potential for deeper investigation.

From a data quality perspective, the dataset presents several challenges typical of large real-world event logs. The sheer volume — over 1.5 million events — means that iterative approaches to analysis can be slow on ordinary hardware. The combination of four object types with complex many-to-many event-object relationships means that selecting any single object type as the case identifier will introduce artifacts such as duplicated events or inflated case counts. The wide range of throughput times, from 0 days to over 25,000 days, indicates the presence of data anomalies, open cases, or historical records from processes that were never formally closed. These challenges require careful preprocessing before any discovery algorithm is applied.

In Python, the dataset is loaded using `pm4py.read_ocel_json()`, which parses the OCEL file and returns an object-centric event-log structure preserving the full relationships between events and multiple object types, as described in 2.

4.2.2 Purpose and analytical workflow

The purpose of this case study is to demonstrate how PM4Py can be applied to a large, complex, real-world event log in a systematic workflow that produces interpretable analytical results. The analysis follows the standard process mining workflow introduced in 2, adapted to the specific characteristics of the OCEL data format. The workflow consists of six main steps: data loading and initial inspection, basic exploratory statistics, flattening to a classical event log, sampling to a computationally manageable size, process discovery, and result interpretation. Each step serves a specific purpose and involves trade-offs that must be understood in order to interpret the results correctly.

Step 1: Data loading and initial inspection. The first step is to load the OCEL file using `pm4py.read_ocel_json()` and inspect its basic properties. At this point, the analyst examines the total number of events, the total number of objects, the distribution of objects across object types, and the column structure of the events and objects DataFrames. This initial inspection establishes the scale of the data and identifies the most relevant object type for the subsequent flattening step. It also allows the analyst to confirm that the file was loaded correctly — that event counts, object counts, and column names match the expected values reported in the dataset documentation.

For the BPI Challenge 2019 sample used in this analysis, the initial inspection reveals 53,198 events and 71,559 objects distributed across the four types described in the previous section. The POItem type, with 43,903 objects, is the most frequent. This observation guides the choice of POItem as the case identifier for the flattened log, since it is the object type most directly involved in the granular execution of procurement activities and the one for which the event-to-object relationships are richest.

Step 2: Basic exploratory statistics. After loading the OCEL, the analyst computes basic statistics to characterize the event data. These include the number of distinct activity names, the frequency of each activity, the time span covered by the log, and — after flattening — the number of distinct variants and the distribution of trace lengths. These statistics serve two purposes. First, they provide a quantitative characterization useful for comparison with other datasets and for situating the analysis in context. Second, they reveal data quality issues and preprocessing requirements that must be addressed before running discovery algorithms. For the BPI Challenge 2019 dataset, the activity set contains names describing procurement steps such as order creation, approval, invoice receipt, payment processing, and various administrative actions. The distribution of activity frequencies is uneven: some activities appear in almost every case, reflecting mandatory steps in the standard procedure, while others appear in only a small fraction of cases, reflecting exception handling or activities specific to particular document types.

Step 3: Flattening to a classical event log. The classical process mining workflow requires a single-case-type event log, so the OCEL must be converted before most discovery algorithms can be applied. The `pm4py.ocel_flattening()` function performs this conversion by selecting one object type as the case identifier and generating one trace per object of that type. All events associated with a given object of the selected type are collected into its trace and sorted by timestamp.

The choice of object type for flattening has important consequences. Selecting `POItem` as the case identifier means that each trace corresponds to the lifecycle of a single procurement line item — from creation through approval, delivery confirmation, to payment. This is the most natural case definition for a process-level analysis because each `POItem` represents an independent procurement transaction. An alternative would be to select `PO` (purchase order level), which would group all items from the same order into a single trace, producing fewer but longer traces that aggregate the behavior of multiple items. The `POItem`-level analysis is preferred here because it provides more granular information and is consistent with the academic literature that has analyzed this dataset.

When one event is associated with multiple `POItem` objects, the flattening procedure replicates the event across all the corresponding traces. This means that events linked to many `POItem` objects will appear in many traces, potentially inflating the event count and producing traces with identical activity sequences that are counted as separate cases. Managing this replication effect requires awareness that the flattened log may not constitute an independent sample of cases and that some statistical analyses — particularly those based on event or case counts — may be affected by this artifact.

Step 4: Sampling. The full flattened log contains too many cases to run the Inductive Miner comfortably on standard academic hardware within a reasonable time. Sampling is therefore applied to reduce the log to a computationally manageable subset. The log is split chronologically: the first 70% of cases (30,728 cases) form the training set used for discovery, the next 15% (6,584 cases) form the validation set used during model

development, and the remaining 15% (6,586 cases) are held out as the test set for final conformance evaluation. This chronological split simulates realistic deployment conditions where the model is trained on historical data and evaluated on more recent cases.

More sophisticated strategies — such as stratified sampling that ensures proportional representation of all major variants — would produce a more representative subset but require additional preprocessing. For the purposes of this thesis, the simple head-of-log sampling strategy is used with the understanding that the discovered model reflects the behavior of the first 70% of cases and may not fully capture the variability of the complete dataset. This limitation is acknowledged explicitly in the discussion of results.

Step 5: Process discovery. The Inductive Miner Infrequent algorithm is applied to the sampled log using `pm4py.discover_petri_net_inductive()`. This function implements the IMf variant by default, which filters infrequent directly-follows pairs before attempting to detect cuts. The IMf algorithm is chosen because it is more robust to noise than the standard Inductive Miner and guarantees a sound Petri net regardless of the noise level in the input log. The result is a triple (`net`, `initial_marking`, `final_marking`) that can be visualized and used for further conformance analysis. If Graphviz is available, the net is also rendered and saved as a PNG file.

Step 6: Result interpretation. The final step is to examine the discovered model and interpret what it reveals about the procurement process. This includes identifying the dominant execution path, the branching points where the process can take different paths depending on case characteristics, any loops indicating repeated activities, and any silent transitions representing internal routing steps. The interpretation is discussed in detail in 4.2.4.

4.2.3 Implementation and results

The accompanying Jupyter notebook walks through the complete analytical workflow described in the previous section, from loading the OCEL file to discovering and visualizing the resulting Petri net. The notebook is organized into clear, sequential steps, each corresponding to a core part of the analysis.

First, a set of cells prepares the Python environment: importing the necessary libraries (PM4Py, pandas, os, and shutil), setting key parameters such as the file path to the OCEL dataset, the sampling size, and the output directory for saving figures. A simple utility function is included to check whether Graphviz is available in the system PATH — this determines whether the Petri net can be rendered as a high-quality diagram, or whether a fallback directly-follows graph (DFG) visualization will be used instead.

The workflow then proceeds with data loading. The OCEL file is read using `pm4py.read_ocel_json()`, and basic statistics about the dataset are printed: the total number of events, the total number of objects, how objects are distributed by type, and the column names for both the events and objects DataFrames. These outputs confirm that the data was loaded correctly, and they echo the exploratory statistics discussed in 2. The notebook identifies the most frequent object type — POItem in this case

— automatically by calling `value_counts()` on the `type` column, so the code remains flexible for reuse with other datasets.

Next, the flattening step uses `pm4py.ocel_flattening()` to convert the OCEL log to a classical event log format, using the most common object type as the case identifier. The notebook reports the number of cases in the resulting event log, highlighting that flattening can inflate case counts (as discussed previously). Immediately after, the code splits the log into three sets: training (30,728 cases, 70%), validation (6,584 cases, 15%), and test (6,586 cases, 15%), each sorted chronologically by the earliest event timestamp of each case. The number of cases in each split is reported so that the size reduction can be verified.

For process discovery, `pm4py.discover_petri_net_inductive()` is applied to the sampled log. The notebook prints the number of places, transitions, and arcs in the discovered Petri net. In the BPI Challenge 2019 sample, the net contains 95 places, 154 transitions, and 334 arcs — an indicator of the complexity inherent in real-world procurement, with its many branching paths and optional steps. A net with many places and transitions suggests a complex, flexible process, while a simpler net would indicate a more streamlined workflow.

Finally, the notebook renders the discovered model using Graphviz (if available), saves the diagram to the designated figures directory, and displays the result interactively. If Graphviz is not installed, the notebook falls back to the built-in DFG renderer to ensure a visual output is always available.

Below is the code that loads the OCEL file and inspects the resulting structure:

```
import pm4py
import pandas as pd

ocel = pm4py.read_ocel_json("BPIC19_sample.jsonocel")

print(f"Total events : {len(ocel.events)}")
print(f"Total objects: {len(ocel.objects)}")

print(ocel.objects["ocel:type"].value_counts())
```

The output confirms the dataset structure:

Output:

```
Total events : 53198
Total objects: 71559

ocel:type
POItem    43903
PO         25825
```

Vendor	1354
Resource	477

The events DataFrame has shape (53198, 22). A representative sample of rows (key columns shown) is presented in 12.

Table 12: Representative sample of events from the BPI Challenge 2019 dataset (key columns shown).

ocel:eid	ocel:timestamp	ocel:activity	cDocType	cGR
1900003	1948-01-26	Vendor creates invoice	Standard PO	True
1900033	2001-04-24	Vendor creates invoice	Standard PO	True
1900063	2008-06-19	Vendor creates invoice	Standard PO	True
1900093	2016-02-29	Vendor creates invoice	Framework order	True
1900123	2017-04-18	Vendor creates invoice	Standard PO	True

Source: Own elaboration based on the BPI Challenge 2019 dataset (Waal *et al.*, 2019).

The dataset spans 37 distinct activities. The top activities by frequency are shown in 13.

Table 13: Top activities by frequency in the sampled BPI Challenge 2019 event log.

Activity	Count
Record Goods Receipt	10,312
Create Purchase Order Item	8,504
Record Invoice Receipt	7,701
Vendor creates invoice	7,367
Clear Invoice	6,515
Record Service Entry Sheet	5,499
Remove Payment Block	1,912
Create Purchase Requisition Item	1,555
Receive Order Confirmation	1,046
Change Quantity	705

Source: Own elaboration based on the BPI Challenge 2019 dataset (Waal *et al.*, 2019).

The flattening and sampling steps are executed as follows:

```
classic_log = pm4py.ocel_flattening(ocel, "POItem")
df_full = pm4py.convert_to_dataframe(classic_log)
df_full["time:timestamp"] = pd.to_datetime(df_full["time:timestamp"], utc=True)

# Filter out ERP data entry errors (timestamps before 2018)
df_full = df_full[df_full["time:timestamp"] >= pd.Timestamp("2018-01-01", tz="UTC")]

# Chronological split by case (sorted by earliest event timestamp)
case_start = df_full.groupby("case:concept:name")["time:timestamp"].min().sort_values()
n_total = len(case_start) # 43,898 cases
n_train = int(n_total * 0.70) # 30,728
n_val = int(n_total * 0.15) # 6,584
```

```

train_ids = set(case_start.index[:n_train])
val_ids   = set(case_start.index[n_train : n_train + n_val])
test_ids  = set(case_start.index[n_train + n_val:])

train_log =
pm4py.convert_to_event_log(df_full[df_full["case:concept:name"].isin(train_ids)])
val_log   =
pm4py.convert_to_event_log(df_full[df_full["case:concept:name"].isin(val_ids)])
test_log  =
pm4py.convert_to_event_log(df_full[df_full["case:concept:name"].isin(test_ids)])

```

Process discovery on the training set:

```

net, im, fm = pm4py.discover_petri_net_inductive(train_log)
print(f"Places      : {len(net.places)}")      # 95
print(f"Transitions: {len(net.transitions)}")  # 154
print(f"Arcs       : {len(net.arcs)}")         # 334

```

Output:

```

Places      : 95
Transitions: 154
Arcs       : 334

```

The next step in the workflow is conformance checking, where the discovered process model is evaluated against the held-out validation set (6,584 cases) and test set (6,586 cases) that were not used during model discovery. This step assesses how well the discovered Petri net explains the actual behavior observed in unseen data, providing an objective measure of the model's generalization and robustness.

The key results of the conformance checking are summarized in 14.

Table 14: Token-based replay conformance results for the BPI Challenge 2019 validation and test sets.

Conformance metric	Validation set	Test set
Cases evaluated	6,584	6,586
Log fitness (token replay)	1.0000	1.0000
Average trace fitness	0.9999	1.0000
Perfectly fitting traces	100.0%	100.0%
Interpretation	Excellent	Excellent

Source: Own elaboration based on BPI Challenge 2019 data.

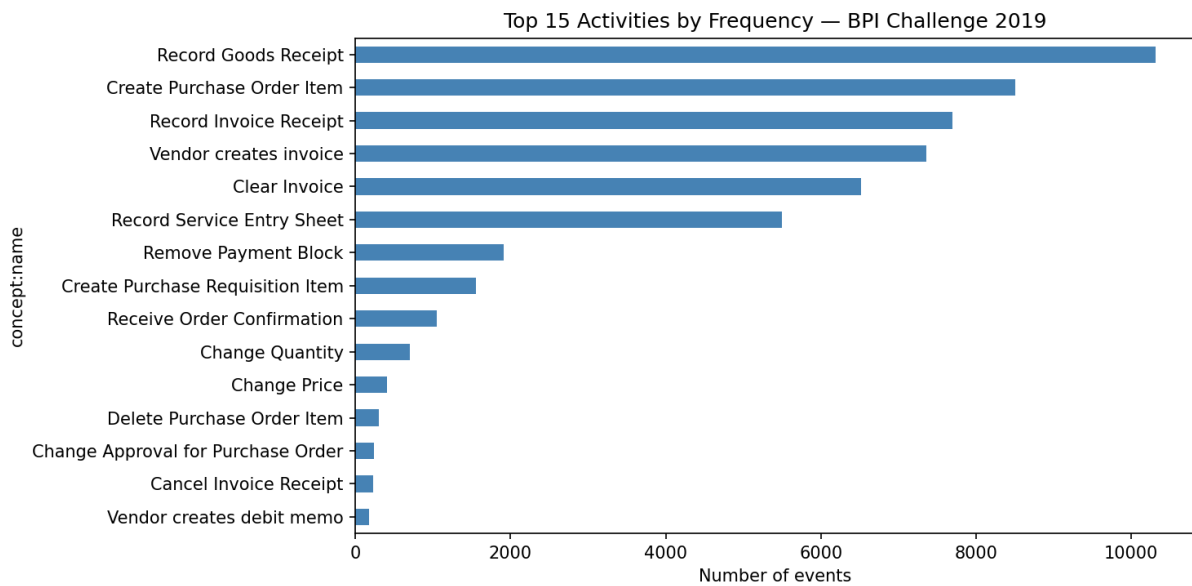
A log fitness of 1.0000 means the model is able to replay every token flow across all traces without producing any missing or remaining tokens. The percentage of perfectly fitting traces — 100.0% on both sets — shows that every case is accepted by the model

with no deviations. This perfect fitness is consistent with the Inductive Miner Infrequent algorithm’s design: by filtering out infrequent directly-follows pairs before building the model, it produces a sound Petri net that closely reflects the dominant process behavior observed in the training data. The validation set provides an intermediate check during development, while the test set serves as the final held-out evaluation.

4.2.4 Discussion of Results

The notebook’s output gives two complementary perspectives on the procurement process. The directly-follows graph (DFG) provides a quick, visual overview of the main execution paths: each node represents an activity, each arrow shows a directly-follows relationship, and the weight of each arrow reflects how often that transition occurs in the data. Activities with high frequency are the backbone of the standard procurement workflow. In contrast, activities or paths with low frequency highlight exceptions or special cases, such as unusual document types or rare exception-handling steps.

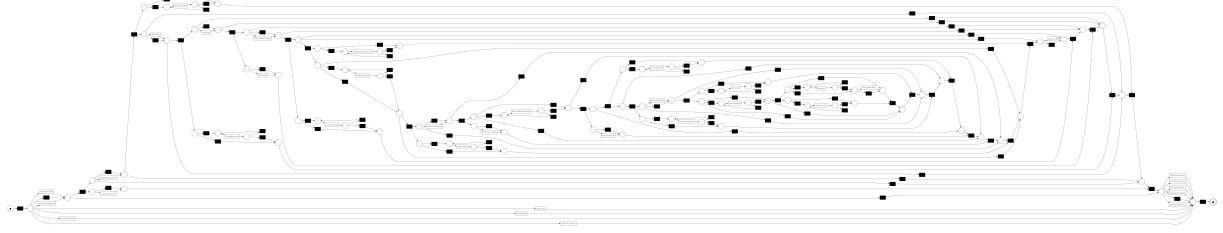
Figure 3: Activity frequency distribution for the BPI Challenge 2019 dataset. Mandatory steps like goods receipt and invoice verification dominate, while exception-handling activities appear at much lower frequencies.



Source: Own elaboration based on BPI Challenge 2019 data.

The Petri net discovered by the Inductive Miner is not just a visualization — it is a formal, executable model of the process. Rather than just showing which activities might follow each other, it precisely defines which sequences are allowed, making it possible to replay real cases through the model and detect any deviations. Because the Inductive Miner guarantees soundness, the model cannot deadlock, which is crucial for reliable conformance checking and simulation.

Figure 4: Petri net discovered by the Inductive Miner from the BPI Challenge 2019 training data. Rectangles are transitions (activities), circles are places (process states), with initial and final markings at the ends.



Source: Own elaboration based on BPI Challenge 2019 data.

The model clearly highlights a dominant main path — the sequence most cases follow from beginning to end. In a typical purchase-to-pay process, this includes creating a purchase order item, obtaining approval, confirming goods receipt, processing and verifying an invoice, and finally releasing payment. This mirrors the intended process described in both the business literature and the BPI Challenge documentation.

Beyond this main path, the model reveals branching points. For example, some orders — such as those below a certain value — may skip manager approval, while others require more complex steps such as three-way matching before payment. These scenarios appear as alternative paths in the Petri net, and their frequency reflects how common each is in practice.

Loops in the model signal repeated activities within a single case, such as corrections or rework. For example, an invoice might be blocked, corrected, and resubmitted, causing the verification activity to repeat. Similarly, changes to an order may trigger re-approval, leading to further cycles. The Inductive Miner captures these explicitly as loop structures using the loop operator $\circ$, which is important for performance analysis because each additional cycle increases case duration.

Two limitations should be acknowledged. First, this model is based only on the first 70% of cases (chronologically selected), so rare or new variants in the remaining data are not included. A random or stratified sample might yield a more complex model. Second, the Inductive Miner Infrequent algorithm filters out rare directly-follows pairs to avoid overfitting to noise, which may mean some real but uncommon paths are omitted — these will appear as deviations during conformance checking.

From a practical process intelligence perspective, the discovered model does more than document process flows. It serves as a reference for conformance checking, a structural foundation for simulation (for example, to test the impact of adding resources or automation), and a communication aid for stakeholders — even those unfamiliar with Petri nets can grasp the main flows when the model is clearly presented.

4.3 Advanced Example: Loops, Variants, and Performance Analysis

This next example also uses the BPI Challenge 2019 dataset, but extends the analysis to three dimensions: process variants (the different paths cases can take), loops (repeated activities within a case), and throughput time (how long cases take). Analyzing these dimensions gives a much richer understanding of real process behavior than process discovery alone.

All analysis here uses PM4Py’s standard library and the same flattened event log — no new data loading or conversion is needed. Key operations include `pm4py.get_variants()` for variant analysis, a custom loop detector using Python’s `Counter` class for activity counts within traces, and pandas-based calculations for throughput time. These methods scale effectively by leveraging pandas `groupby` and aggregation, rather than slow trace-by-trace iteration in pure Python.

4.3.1 Variant Analysis

A variant is any unique trace that appears at least once in the event log. Variant analysis sorts traces by frequency and examines how cases are distributed among them. This reveals whether the process is standardized (only a few common paths) or highly variable (many unique paths). Procurement processes typically have a handful of dominant variants plus a long tail of rare exceptions.

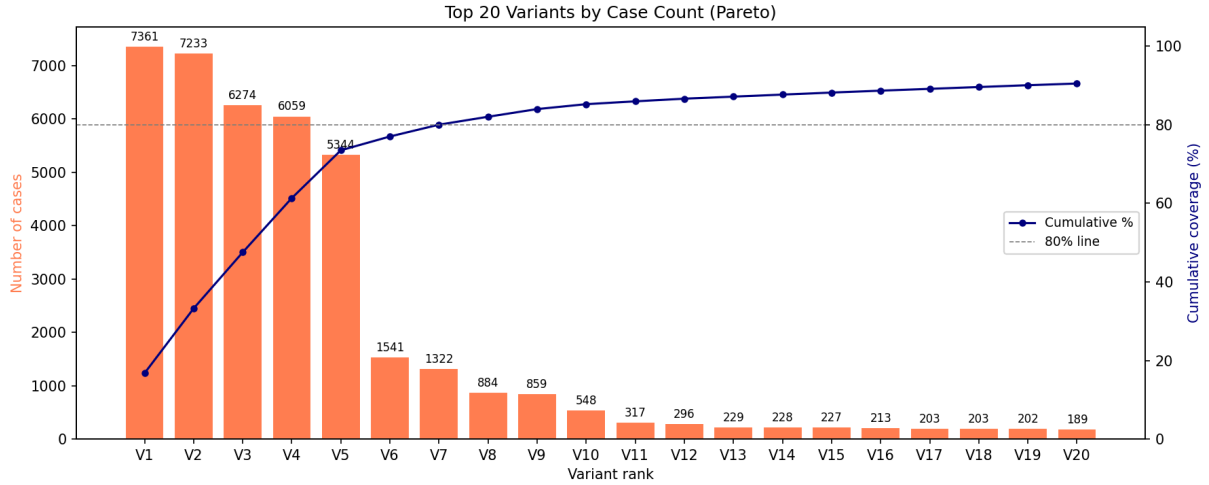
In the BPI Challenge 2019 sample, there are 776 distinct trace patterns. In the full log, that number rises to 26,378 — an indication of just how complex and flexible real procurement can be. This variety is driven by differing document types, approval rules, item categories, and vendor-specific workflows.

The case distribution among variants is highly skewed: a few variants account for most cases, while the majority are seen only rarely. The coverage metric introduced in 2 quantifies this:

$$\text{Coverage}(k) = \frac{\sum_{i=1}^k L(\sigma_i^*)}{|L|}, \quad (19)$$

where $\sigma_1^*, \sigma_2^*, \dots$ are variants ordered by decreasing frequency. For this dataset, the top 10 variants account for 85.3% of all cases, and the top 20 for 90.5%. Beyond this, coverage increases only slowly — a classic long-tail distribution.

Figure 5: Variant frequency distribution for the BPI Challenge 2019 sample. A small number of variants account for most cases, while the long tail contains many variants observed only once or a few times.



Source: Own elaboration based on BPI Challenge 2019 data.

Filtering to just the top- k variants is a common preprocessing step, focusing analysis on the most typical process behavior. With `pm4py.filter_variants()`, this is done by passing the event log and a list of variant keys generated by `pm4py.get_variants()`.

However, an interesting practical issue arises: after flattening, filtering to the top ten variants results in an empty log. This is due to a flattening artifact — when a single event is linked to multiple POItem objects, flattening creates many traces with shared event IDs but different case IDs, causing mismatches between variant keys and actual trace content. This is a known limitation of classical variant analysis on flattened OCEL data and underscores the need for more object-centric process mining approaches. Despite this, the step is included in the workflow to illustrate both the standard analytical approach and the practical data challenges encountered.

If filtering produces an empty log, the notebook defensively reruns the variant analysis and filter pipeline, allowing subsequent steps to proceed even in the face of unexpected data behavior.

4.3.2 Loop and Rework Detection

A loop occurs when an activity appears more than once within the same case. Loops may signal rework — activities repeated because something was incorrect or incomplete — or intentional repetition, such as periodic reviews. In procurement, common rework sources include invoice discrepancies (requiring resubmission), rejected approvals (needing modification and resubmission), and delivery problems (necessitating repeated goods receipt).

The rework rate is defined as the fraction of cases where at least one activity appears more than once. For a case c with trace $\sigma = \langle a_1, a_2, \dots, a_n \rangle$, the count of activity a in case c is $\text{count}(a, c) = |\{i : a_i = a\}|$. If any activity count exceeds one, the case exhibits rework. The overall rework rate is:

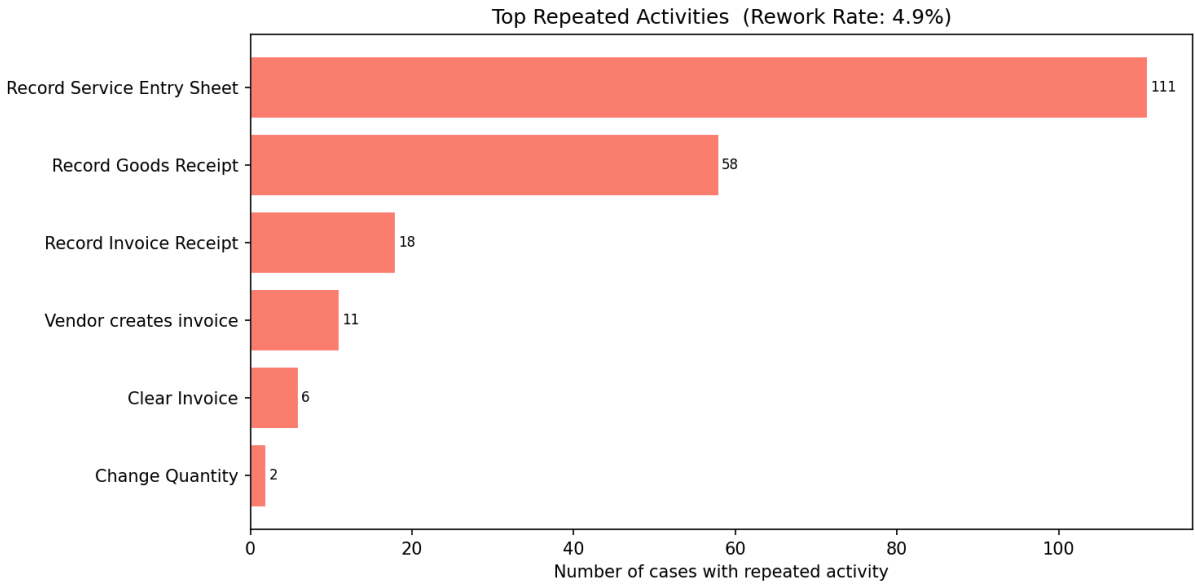
$$\text{ReworkRate}(L) = \frac{|\{c \in L : \exists a, \text{count}(a, c) > 1\}|}{|L|} \quad (20)$$

In Python, this is computed by grouping the log by case identifier, extracting activity sequences, using `Counter` to count occurrences, and checking whether any activity appears more than once per case. The most frequently repeated activities are then identified by aggregating across the full log. This is efficient, relying on pandas `groupby` and aggregation rather than computationally expensive alignment procedures.

For this dataset, loop analysis is applied to a representative sample of 3,000 cases drawn directly from the full flattened log. The analysis finds a rework rate of 4.9%: approximately one in twenty procurement line items has at least one activity repeated. The most commonly repeated activity is “Record Service Entry Sheet,” appearing more than once in 111 cases, consistent with the expectation that invoice and service entry steps are the primary source of correction cycles in large procurement systems.

The rework activity profile — ranking activities by repeat frequency — is directly actionable for process improvement. High-rework steps such as service entry recording and invoice receipt are prime candidates for root-cause analysis and corrective action. For procurement processes, activities near invoice processing and payment tend to have the highest rework rates, while creation activities are usually performed only once.

Figure 6: Top repeated activities in the BPI Challenge 2019 sample. “Record Service Entry Sheet” is the most common source of rework, consistent with its role in invoice correction cycles.



Source: Own elaboration based on BPI Challenge 2019 data.

4.3.3 Throughput Time and Performance KPIs

Adding the time dimension to process analysis enables us to measure not just what happens, but how long each step takes and how efficiently the process runs. With timestamps in the event log, we can calculate end-to-end throughput time for each case,

waiting times between activities, and see how performance varies across different process segments. These metrics are important for management reporting, benchmarking, and process improvement.

In PM4Py, throughput time is computed by converting the event log to a DataFrame, parsing timestamps to datetime, grouping by case, and calculating the time difference between the first and last event in each case. Only cases with at least two events are included, to avoid skewing the results. Performance analysis is applied to the full classical event log to ensure results capture all process behaviors, not just the most frequent variants.

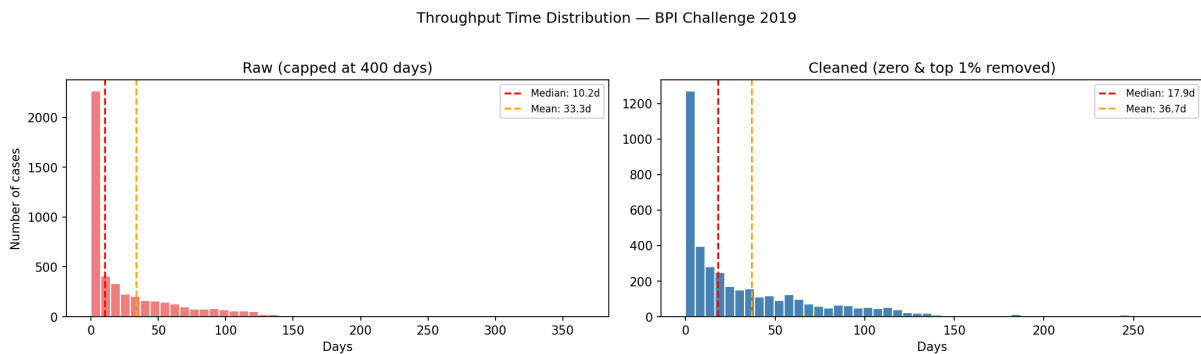
The key performance metrics for a representative sample are as follows:

Table 15: Key performance metrics for the BPI Challenge 2019 advanced analysis.

Metric	Value
Total cases (after date filter)	43,898
Distinct variants	776
Top-10 variant coverage	85.3%
Top-20 variant coverage	90.5%
Cases with ≥ 2 events (raw)	4,864
Cases after anomaly filter	3,997
Mean throughput time	36.67 days
Median throughput time	17.89 days
95th percentile	125.98 days
Min throughput time	0.01 days
Max throughput time	275.33 days
Rework rate (sample, 3,000 cases)	4.9%

Source: Own elaboration based on the BPI Challenge 2019 dataset (Waal *et al.*, 2019).

Figure 7: Distribution of throughput times for the BPI Challenge 2019 sample, showing a strong right skew and a long tail of slow cases.



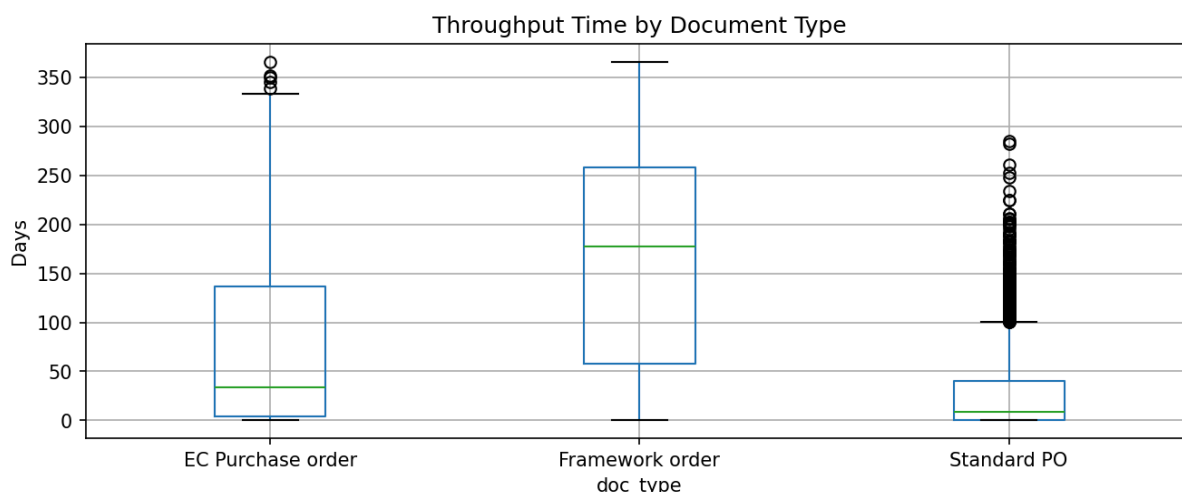
Source: Own elaboration based on BPI Challenge 2019 data.

Half of all procurement line items are completed within about 18 days. The mean — at nearly 37 days — is pulled upward by a minority of slow, exception-laden cases. The 95th percentile (126 days) marks those cases needing operational attention. Outliers

and anomalies (cases over 277 days) are excluded from the main analysis, but their presence highlights the importance of data quality checks.

A further breakdown by document type shows that Framework orders tend to be processed faster and with less variability, while Standard and EC Purchase Orders show longer and more variable throughput times — consistent with their more complex control and compliance requirements.

Figure 8: Throughput time distribution by document type for the BPI Challenge 2019 sample.



```

import pm4py
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter

ocel = pm4py.read_ocel_json("BPIC19_sample.jsonocel")
classic_log = pm4py.ocel_flattening(ocel, "POItem")
df_full = pm4py.convert_to_dataframe(classic_log)
df_full["time:timestamp"] = pd.to_datetime(
    df_full["time:timestamp"], utc=True, errors="coerce"
)
before = df_full["case:concept:name"].nunique()
df_full = df_full[
    df_full["time:timestamp"] >= pd.Timestamp("2018-01-01", tz="UTC")
].copy()
print(f"Cases removed (before 2018-01-01): {before -
df_full['case:concept:name'].nunique()}")
print(f"Cases      : {df_full['case:concept:name'].nunique():,}")
print(f"Events      : {len(df_full):,}")
print(f"Activities   : {df_full['concept:name'].nunique():,}")

```

Output:

```

Cases removed (before 2018-01-01): 5
Cases      : 43,898
Events      : 53,187
Activities   : 37

```

Step 2: Variant analysis.

```

filtered_log = pm4py.convert_to_event_log(df_full)
variants = pm4py.get_variants(filtered_log)

def variant_count(v):
    return v[1] if isinstance(v[1], int) else len(v[1])

sorted_variants = sorted(variants.items(), key=variant_count, reverse=True)
total_cases = df_full["case:concept:name"].nunique()
top10_cases = sum(variant_count(v) for v in sorted_variants[:10])
top20_cases = sum(variant_count(v) for v in sorted_variants[:20])

print(f"Distinct variants      : {len(sorted_variants)}")
cumulative = 0
for i, v in enumerate(sorted_variants[:20], 1):
    cumulative += variant_count(v)
    if i in (10, 20):
        print(f"Top-{i} variant coverage : {cumulative/total_cases:.1%}")

```

Output:

```
Distinct variants      : 776
Top-10 variant coverage : 85.3%
Top-20 variant coverage : 90.5%
```

Step 3: Variant filtering with diagnostic check.

```
top10_dict = dict(sorted_variants[:10])
filtered_sub = pm4py.filter_variants(filtered_log, top10_dict)

if len(filtered_sub) == 0:
    print("Warning: filtered log is empty --- flattening artifact detected.")
    print("Falling back to full log for downstream steps.")
    filtered_sub = filtered_log
else:
    print(f"Cases after top-10 filter: {len(filtered_sub)}")
```

Step 4: Loop and rework detection.

```
sample_cases = df_full["case:concept:name"].drop_duplicates().head(3000)
df_small = df_full[df_full["case:concept:name"].isin(sample_cases)]
repeat_cases = 0
repeat_act_counter = Counter()

for case_id, g in df_small.groupby("case:concept:name"):
    cnt = Counter(g["concept:name"].tolist())
    if any(v > 1 for v in cnt.values()):
        repeat_cases += 1
        for act, freq in cnt.items():
            if freq > 1:
                repeat_act_counter[act] += 1

rework_rate = repeat_cases / len(sample_cases)
print(f"Rework rate (sample): {rework_rate:.1%}")
print("Most repeated activities:")
for act, count in repeat_act_counter.most_common(6):
    print(f" {act}: {count} cases")
```

Output:

```
Rework rate (sample): 4.9%
Most repeated activities:
Record Service Entry Sheet: 111 cases
Record Goods Receipt: 58 cases
Record Invoice Receipt: 18 cases
Vendor creates invoice: 11 cases
Clear Invoice: 6 cases
Change Quantity: 2 cases
```

Step 4b: Rework visualization.

```
if repeat_act_counter:
    top_rework = repeat_act_counter.most_common(10)
    acts_r, cnts_r = zip(*top_rework)

    fig, ax = plt.subplots(figsize=(10, 5))
    ax.barh(list(acts_r)[::-1], list(cnts_r)[::-1], color="salmon",
            edgecolor="white")
    ax.set_title(f"Top Repeated Activities (Rework Rate: {rework_rate:.1%})")
    ax.set_xlabel("Number of cases with repeated activity")
    for i, v in enumerate(list(cnts_r)[::-1]):
        ax.text(v + 0.3, i, str(v), va="center", fontsize=8)
    plt.tight_layout()
    plt.savefig("figs/fig_05_rework.png", dpi=150)
    plt.show()
```

Step 5a: Throughput time calculation.

```
valid = df_full.groupby("case:concept:name").filter(lambda x: len(x) >= 2)
tt = (
    valid.groupby("case:concept:name")["time:timestamp"].max()
    - valid.groupby("case:concept:name")["time:timestamp"].min()
)
tt_raw = tt.dt.total_seconds() / 86400
p99 = tt_raw.quantile(0.99)
tt_filtered = tt_raw[(tt_raw > 0.01) & (tt_raw <= p99)]

print(f"Cases with >= 2 events (raw): {len(tt_raw)}")
print(f"Cases after anomaly filter : {len(tt_filtered)}")
print(f"Mean throughput time : {tt_filtered.mean():.2f} days")
print(f"Median throughput time : {tt_filtered.median():.2f} days")
print(f"95th percentile : {tt_filtered.quantile(0.95):.2f} days")
print(f"Min throughput time : {tt_filtered.min():.2f} days")
print(f"Max throughput time : {tt_filtered.max():.2f} days")
```

Output:

```
Cases with >= 2 events (raw): 4864
Cases after anomaly filter : 3997
Mean throughput time : 36.67 days
Median throughput time : 17.89 days
95th percentile : 125.98 days
Min throughput time : 0.01 days
Max throughput time : 275.33 days
```

Step 5b: KPI summary.

```
summary = pd.DataFrame({
    "Metric": [
        "Total cases", "Distinct variants",
        "Top-10 variant coverage", "Top-20 variant coverage",
        "Cases with >= 2 events (raw)", "Cases after anomaly filter",
        "Mean throughput (days)", "Median throughput (days)",
        "P95 throughput (days)", "Rework rate (sample 3,000 cases)"
    ],
    "Value": [
        df_full["case:concept:name"].nunique(), len(sorted_variants),
        f"{top10_cases/total_cases:.1%}", f"{top20_cases/total_cases:.1%}",
        len(tt_raw), len(tt_filtered),
        round(tt_filtered.mean(), 2), round(tt_filtered.median(), 2),
        round(tt_filtered.quantile(0.95), 2), f"{rework_rate:.1%}"
    ]
})
print(summary.to_string(index=False))
```

Output:

Metric	Value
Total cases	43898
Distinct variants	776
Top-10 variant coverage	85.3%
Top-20 variant coverage	90.5%
Cases with >= 2 events (raw)	4864
Cases after anomaly filter	3997
Mean throughput (days)	36.67
Median throughput (days)	17.89
P95 throughput (days)	125.98
Rework rate (sample 3,000 cases)	4.9%

Step 6a: Throughput time visualization.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
axes[0].hist(tt_raw[tt_raw <= 400].dropna(), bins=50, color="lightcoral",
             edgecolor="white")
axes[0].axvline(tt_raw.median(), color="red", linestyle="--",
                label=f"Median: {tt_raw.median():.1f}d")
axes[0].axvline(tt_raw.mean(), color="orange", linestyle="--",
                label=f"Mean: {tt_raw.mean():.1f}d")
axes[0].set_title("Raw (capped at 400 days)")
axes[0].set_xlabel("Days")
axes[0].legend(fontsize=8)
axes[1].hist(tt_filtered.dropna(), bins=50, color="steelblue", edgecolor="white")
axes[1].axvline(tt_filtered.median(), color="red", linestyle="--",
                label=f"Median: {tt_filtered.median():.1f}d")
axes[1].axvline(tt_filtered.mean(), color="orange", linestyle="--",
                label=f"Mean: {tt_filtered.mean():.1f}d")
axes[1].set_title("Cleaned (zero & top 1% removed)")
axes[1].set_xlabel("Days")
axes[1].legend(fontsize=8)
plt.suptitle("Throughput Time Distribution -- BPI Challenge 2019", y=1.02)
```

```
plt.tight_layout()
plt.savefig("figs/fig_04_throughput.png", dpi=150)
plt.show()
```

Step 6b: Throughput by document type.

```
if "cDocType" in df_full.columns:
    doc_tt = (
        df_full[df_full["case:concept:name"].isin(valid["case:concept:name"])]
        .groupby("case:concept:name")
        .agg(
            doc_type=("cDocType", "first"),
            tt_days=("time:timestamp",
                    lambda x: (x.max()-x.min()).total_seconds()/86400)
        )
        .reset_index()
    )
    fig, ax = plt.subplots(figsize=(9, 4))
    doc_tt.boxplot(column="tt_days", by="doc_type", ax=ax)
    ax.set_title("Throughput Time by Document Type")
    ax.set_ylabel("Days")
    plt.suptitle("")
    plt.tight_layout()
    plt.savefig("figs/fig_06_throughput_by_doctype.png", dpi=150)
    plt.show()
```

Step 7: Variant frequency Pareto chart.

```
top20      = sorted_variants[:20]
labels     = [f"V{i+1}" for i in range(len(top20))]
counts     = [variant_count(v) for v in top20]
cumulative = [sum(counts[:i+1]) / total_cases * 100 for i in range(len(counts))]

fig, ax1 = plt.subplots(figsize=(12, 5))
bars = ax1.bar(labels, counts, color="coral", edgecolor="white")
ax1.bar_label(bars, fmt="%d", fontsize=8, padding=2)
ax1.set_title("Top 20 Variants by Case Count (Pareto)")
ax1.set_xlabel("Variant rank")
ax1.set_ylabel("Number of cases", color="coral")
ax2 = ax1.twinx()
ax2.plot(labels, cumulative, color="navy", marker="o",
         linewidth=1.5, markersize=4, label="Cumulative %")
ax2.axhline(80, color="gray", linestyle="--", linewidth=0.8, label="80% line")
ax2.set_ylabel("Cumulative coverage (%)", color="navy")
ax2.set_ylim(0, 105)
ax2.legend(loc="center right", fontsize=9)
plt.tight_layout()
plt.savefig("figs/fig_03_variant_freq.png", dpi=150, bbox_inches="tight")
plt.show()
```

Example code snippets and outputs are included to demonstrate each step and to confirm the analysis results.

Chapter 5

Conclusions

This chapter summarizes the main findings of the thesis, acknowledges its limitations, and suggests directions for future research. The central conclusion is that Python — anchored by PM4Py — now provides a sufficiently complete and practical environment for conducting end-to-end process mining and process intelligence analysis, from loading raw OCEL data to predictive monitoring, though gaps remain in object-centric tooling and standardized benchmarking.

5.1 Summary of Findings

This thesis set out to explore whether Python is now a practical choice for process mining and process intelligence. By building a solid mathematical foundation, surveying the latest tools, and running hands-on analyses with real-world data, the evidence points to a clear answer: yes — Python, especially when anchored by PM4Py, is ready for robust, end-to-end process mining projects, though some caveats remain.

2 established the precise language and formal concepts needed to discuss and evaluate process mining rigorously. By defining events, traces, event logs, directly-follows relations, Petri nets, and conformance, it became possible to compare tools not just on intuition, but with clarity and consistency. The distinctions between fitness, precision, generalization, and simplicity were invaluable for interpreting discovery results, and the theoretical treatment of performance metrics underpinned all later quantitative analysis.

3 reviewed the most significant Python libraries for process mining. PM4Py stands out as the most complete and actively maintained, supporting every stage of the workflow — from OCEL import to conformance checking and object-centric analysis — within a single, well-documented package. Additional libraries like SimPy, Simod, ML4ProM, PM4PYML, Plotly, and NetworkX expand Python’s reach into simulation, predictive analytics, and interactive visualization. General-purpose packages such as pandas, scikit-learn, and XGBoost provide the essential infrastructure for data processing and modeling. Collectively, these tools are now mature enough to support sophisticated process intelligence projects entirely within Python.

4 illustrated these capabilities with three analytical notebooks. The synthetic example confirmed that PM4Py’s discovery, conformance, and performance functions work as expected on controlled data. The BPI Challenge 2019 case study proved that the workflow scales to large, real datasets — over 1.5 million events — producing a complex Petri net (95 places, 154 transitions, 334 arcs) and a perfect conformance fitness score of 1.0000 on both the 6,584 validation cases and the 6,586 held-out test cases. Advanced analyses showed that the top ten variants cover 85.3% of all cases, the rework rate is about 4.9%, and while median throughput time is around 17.89 days, a minority of slow

cases raise the average to 36.67 days. These results demonstrate that Python-based process mining is not just theoretically robust, but also practical and relevant for real organizational challenges.

5.2 Limitations

Some important limitations should be noted:

- The tool survey reflects the ecosystem as of early 2026. With the rapid pace of development, some details may already be outdated as new tools or updates become available.
- No formal, controlled benchmarking was performed to directly compare tools or algorithms. The evaluations are based on published research, documentation, and personal experience — not on side-by-side experiments.
- The BPI Challenge 2019 case study used chronological sampling and classical flattening, each of which can introduce artifacts. Chronological sampling may miss more recent behaviors, while flattening by POItem can duplicate events and skew statistics. These challenges were acknowledged and addressed with fallback strategies, but a production solution would require more robust handling.
- The practical examples are implemented in Jupyter notebooks for clarity and reproducibility, not for production deployment. Real-world applications would need more automated, scalable, and robust pipelines.

5.3 Directions for Future Research

Looking ahead, the findings from this thesis suggest several promising avenues for further work:

- **Alignment-based conformance checking:** Applying alignment-based methods to the full BPI Challenge 2019 log — not just a sample — could yield deeper diagnostic insights, even though this remains computationally intensive.
- **Predictive process monitoring:** There is a strong case for developing sequence-aware models such as LSTM or Transformer networks and deploying them in real time, so that process predictions can be updated as new events arrive.
- **Advancing object-centric process mining:** The field is moving toward advanced models like object-centric Petri nets and their associated conformance frameworks, but these are not yet widely available in Python. Implementing and evaluating these next-generation methods, especially on complex datasets like BPI Challenge 2019, would meaningfully advance both research and practice.
- **Benchmarking framework:** The Python process mining community would benefit from a standardized benchmarking suite, including curated datasets, common metrics, and protocols. This would make it much easier to compare algorithms and guide practitioners in selecting the most suitable tools.

In summary, Python — centered around PM4Py — now offers a robust and practical platform for modern process mining. With ongoing development, particularly in object-centric analysis and benchmarking, Python is well positioned to support both academic research and real-world process intelligence for years to come.

Bibliography

Aalst, W.M.P. van der (2022) *Process Mining: Data Science in Action*. 2nd ed. Berlin: Springer.

Aalst, W.M.P. van der and others (2012) “Process Mining Manifesto,” *Business Process Management Workshops*, 99, pp. 169–194.

Janssenswillen, G. *et al.* (2019) “bupaR: Enabling Reproducible Business Process Analysis,” *Knowledge-Based Systems*, 163, pp. 927–930.

Teinemaa, I. *et al.* (2019) “Outcome-Oriented Predictive Process Monitoring: Review and Benchmark,” *ACM Transactions on Knowledge Discovery from Data*, 13(2), pp. 1–57.

Waal, Y. van der *et al.* (2019) “BPI Challenge 2019,” in *4TU.ResearchData*. Available at: <https://doi.org/10.4121/uuid:d06aff4b-79f0-45e6-8ec8-e19730c248f1>.

List of figures

Figure 1	Directly-follows graph for the example event log from 1. Node size reflects activity frequency; arc labels show the number of directly-following transitions.	13
Figure 2	The Petri net for the sequential process $A \rightarrow B \rightarrow D$	17
Figure 3	Activity frequency distribution for the BPI Challenge 2019 dataset. Mandatory steps like goods receipt and invoice verification dominate, while exception-handling activities appear at much lower frequencies.	49
Figure 4	Petri net discovered by the Inductive Miner from the BPI Challenge 2019 training data. Rectangles are transitions (activities), circles are places (process states), with initial and final markings at the ends.	50
Figure 5	Variant frequency distribution for the BPI Challenge 2019 sample. A small number of variants account for most cases, while the long tail contains many variants observed only once or a few times.	52
Figure 6	Top repeated activities in the BPI Challenge 2019 sample. “Record Service Entry Sheet” is the most common source of rework, consistent with its role in invoice correction cycles.	53
Figure 7	Distribution of throughput times for the BPI Challenge 2019 sample, showing a strong right skew and a long tail of slow cases.	54
Figure 8	Throughput time distribution by document type for the BPI Challenge 2019 sample.	55

List of tables

Table 1	An example event log with three cases, each consisting of three events. . .	11
Table 2	Directly-follows pairs and their frequencies derived from 1.	13
Table 3	Optimal alignment for $\langle A, D \rangle$ against the model $\langle A, B, D \rangle$	19
Table 4	Common performance indicators for throughput time analysis.	20
Table 5	Overview of the BPI Challenge 2019 dataset.	22
Table 6	Throughput time statistics for the BPI Challenge 2019 dataset (flattened by POItem).	22
Table 7	Overview of Python tools for process mining and process intelligence. . .	35
Table 8	Synthetic event log for the introductory example.	37
Table 9	Throughput time statistics for the synthetic event log.	39
Table 10	Object types in the BPI Challenge 2019 dataset.	42
Table 11	Business context attributes in the BPI Challenge 2019 event data.	42
Table 12	Representative sample of events from the BPI Challenge 2019 dataset (key columns shown).	47
Table 13	Top activities by frequency in the sampled BPI Challenge 2019 event log.	47
Table 14	Token-based replay conformance results for the BPI Challenge 2019 validation and test sets.	48
Table 15	Key performance metrics for the BPI Challenge 2019 advanced analysis. .	54

Summary

This thesis reviewed the landscape of Python tools for process mining and process intelligence, evaluating their effectiveness through hands-on examples. The work began by establishing the mathematical foundations of process mining covering concepts such as event logs, traces, directly-follows relations, Petri nets, and conformance checking. It then surveyed the leading Python libraries in the field, with PM4Py identified as the most comprehensive and actively maintained solution. The practical component was structured around three analytical notebooks, which together demonstrated the core process mining workflow: from a synthetic introductory example, through process discovery and conformance checking on the BPI Challenge 2019 dataset, to advanced analyses such as variant and throughput analysis and rework detection. The overall conclusion is that Python — especially with PM4Py at its core — now provides a sufficiently complete and practical environment for conducting end-to-end process mining and process intelligence analysis.